

Module 1: Lesson 1

Introduction to Machine Learning



Outline

- ▶ Machine Learning
- ▶ Time Series Momentum
- ▶ The Linear Regression model

Machine Learning (1/3): AI, ML, and DL

When we mention the concepts of Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL), we are referring to different things.

- ▶ **AI** represents the effort to automate intellectual tasks normally performed by humans. Some of these tasks involve learning; others do not.

Consider a chess program that, using nested "if" loops, determines what action to undertake under each possible location of the pieces on the board. This program does not involve learning (may not even be considered AI anymore).

- ▶ **ML** deals with computer programs that allow computers to learn from data. Humans input data as well as the answers expected from (some part) of the data, while the program dictates the rules.

A computer program is said to learn from experience with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience.

- ▶ **DL** is a strategy for learning representations from data that learns successive layers of increasingly meaningful representations. Neural networks are the most usual example of DL models.

Machine Learning (2/3): Supervised and Unsupervised learning

At the most fundamental level, ML can be categorized into two main types: Supervised Learning and Unsupervised Learning.

- ▶ **Supervised Learning** models the relationship between measured features of the data and some labels. Once the model is trained and determined, it can be applied to other data with unknown labels to make predictions.

Supervised learning is further subdivided into classification tasks and regression tasks.

- ▶ **Unsupervised Learning** involves modeling the features of a dataset without reference to any label and is often described as "letting the dataset speak for itself."

These models include tasks such as clustering and dimensionality reduction.

Machine Learning (3/3): Elements of a supervised ML model

In supervised learning, the data is represented by some labels associated with certain features.

In supervised learning, the data is represented by some labels associated with certain features.

- ▶ Features matrix (X): Composed of instances (rows) that have some input features (columns).
- ▶ Labels (also targets or output, y): A vector with the information we want the model to predict.

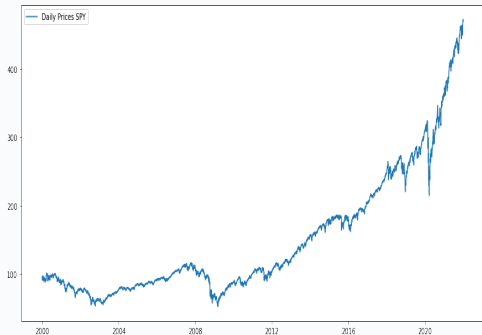
For each instance in X , there is a value, a label, in y (perhaps vector-valued).

The model is a function $g(\cdot)$ that takes the X matrix as input and tries to predict values as close as possible to the y targets.

Training: The computations that are developed to obtain the $g(\cdot)$ function.

When the model is trained, we can make predictions for the labels of data that have been unused in training (test sample).

Time Series Momentum



We are going to study the basic elements of ML outlined above in the context of a simple financial application: Time series momentum.

A "time series" momentum strategy simply buys stocks that had a positive return in the past and sells stocks that had a negative return in the past.

Our ML task in this application is to use a time series momentum strategy that exploits the predictability of SPY, an ETF that tracks the S&P500 index.

This application will illustrate the basic tools of ML and also the complicated task of developing useful predictive models of asset returns.

Linear Regression (1/4): Elements

Linear Regression imposes $g(\cdot)$ to be a linear function of the inputs to predict the labels:

$$\hat{y} = g(X) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

- ▶ $\hat{y}$ is the predicted value for the label.
- ▶ n is the number of features.
- ▶ x_j is the j^{th} input feature.
- ▶ θ_k is the k^{th} model parameter, including the bias term θ_0 .

We can train the vector θ by minimizing the mean-squared error (MSE) of the prediction in the training sample:

$$\min_{\theta} \frac{1}{m} \sum_{i=1}^m \left(y^{(i)} - x^{(i)} \theta \right)^2$$

where m is the number of instances, each represented by a label $y^{(i)}$ and a row vector of features $x^{(i)}$.

Linear Regression (2/4): Training

Minimization of the MSE leads to a closed form expression, called the *Normal Equation*:

$$\hat{\theta} = (X^T X)^{-1} X^T y \quad (1)$$

The computations involved for the inverse part are extremely costly when we have information on a large number of features or too many instances to fit in the memory of the computer.

In Machine Learning applications, we usually opt for a generic optimization algorithm that is capable of finding optimal solutions to a wide range of problems: **Gradient Descent**.

$$\theta^{[next\ step]} = \theta^{[previous\ step]} - \eta \nabla_{\theta} MSE \left(\theta^{[previous\ step]} \right)$$

where $\nabla_{\theta} MSE(\theta)$ is the gradient of the MSE with respect to θ , and η is the **learning rate**.

Linear Regression (3/4): R^2 , In-sample vs. Out-of-sample

To measure the goodness of fit of the model, we can compute the coefficient of determination, or R^2 :

$$R^2 = 1 - \frac{\sum_{i=1}^m (y^{(i)} - x^{(i)} \hat{\theta})^2}{\sum_{i=1}^m (y^{(i)} - \bar{y})^2}$$

where $\bar{y}$ is the average label in the training sample. A low R^2 in the training sample suggests **underfitting**.

The R^2 of the linear regression indicates the proportion of the labels' variance explained by the model.

- We can also understand the R^2 as the prediction gains made by the model against a prediction model that just uses the sample average to predict the labels.

In the training sample (or *in-sample*) the R^2 of the Linear Regression will always be above zero (if there is a bias term).

In the test sample (or *out-of-sample*) the R^2 of Linear Regression may be negative:

- A model may be arbitrarily bad at making predictions out-of-sample, even worse than making a prediction based on the average. A potential solution is to increase the complexity of the model.

Linear Regression (4/4): Overfitting

Machine Learning algorithms may be powerful at predicting the labels of the training sample.

However, the trained model may yield bad results when trying to generalize to new data: **Overfitting**.

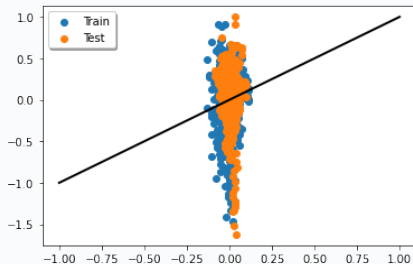
Different ways to avoid overfitting:

- ▶ Gather more training data to reduce noise or clean the data to fix errors or remove outliers.
- ▶ Regularization. Force the trainable parameters to be relatively small and avoid overfitting.
- ▶ Early stopping. Introduce a validation sample to stop training as soon as the model starts to generalize badly.
- ▶ Tuning of hyperparameters. Determine the combination that best fits the trained model in the validation sample.

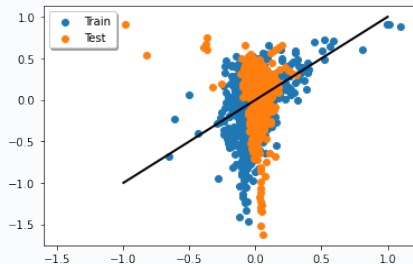
Hyperparameter: a parameter of a learning algorithm that does not explicitly serve to predict the labels. As such, hyperparameters are fixed during training.

Time Series Momentum: Results

Linear Regression:
(x-axis: Predicted return. y-axis: Actual return)



Linear Regression with powers of features:
(x-axis: Predicted return. y-axis: Actual return)



The Linear Regression model clearly underfits the data and is of little use to predict the evolution of SPY.

Increasing the complexity of the model by adding powers of the features leads to overfitting.

Summary of Lesson 1

In Lesson 1, we have looked at:

- ▶ The main distinctions between AI, ML, and DL
- ▶ The two types of Machine Learning models: Supervised and Unsupervised
- ▶ The Linear Regression model
- ▶ The overfitting problem

⇒ **References for this lesson:**

Géron, Aurélien. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 2nd ed., O'Reilly Media, Inc., 2019.

TO DO NEXT: Now, please go to the associated Jupyter Notebook for this lesson to review the concepts of Lesson 1.

In the next lesson, we will describe the main tools for parameter regularization and cross-validation.

Module 1: Lesson 2

Introduction to Machine Learning



Outline

- ▶ Linear Models and regularization
- ▶ Hyperparameter tuning

Linear models and overfitting

Regularization is a good option for reducing the overfitting of an ML model. Regularization means constraining the learned parameters θ and reducing the degrees of freedom of the model to fit the training data.

In linear regression, we can regularize the parameters by minimizing in training a general cost function of the form:

$$J(\theta) = MSE(\theta) + Penalty(\theta)$$

We next describe three methods: Ridge, Lasso, and Elastic Net. Each method specifies different $Penalty(\theta)$ functions to constrain the parameters.

► **Important:** The penalty term should only be added to the cost function during training.

In the validation or test samples, we are only interested in evaluating the unregularized performance of the model, say, the $MSE(\theta)$ in linear regression.

The methods described below are of general application in ML learning problems and are not specific to the Linear Regression framework.

Ridge Regression

Ridge Regression is a *shrinkage* regularization technique that aims at keeping the parameter values of the model as small as possible. The penalty function is of the form:

$$\alpha \sum_{i=1}^n \theta_i^2$$

Notice that we do not aim to regularize the bias term θ_0 . Ridge Regression “forces” the model to become closer to a model where we use the average in training as the prediction.

The hyperparameter α determines how much we wish to regularize the model. A very high α will fit the data with a flat line through the average of the labels y .

Using Gradient Descent with Ridge Regression, the update of the parameters will be given by:

$$\theta^{[next\ step]} = \theta^{[previous\ step]} - \eta \nabla_{\theta} MSE \left(\theta^{[previous\ step]} \right) - a \odot \theta^{[previous\ step]}$$

where a is a column vector of n elements all of which have the value α , except the first term that has a value of 0 and $\odot$ refers to element-by-element multiplication.

Lasso Regression

The Lasso (Least Absolute Shrinkage and Selection Operator) Regression sets to zero the parameters associated with the least important features. That is, Lasso Regression performs a selection of features that are useful for prediction. The regularization term is of the form:

$$\alpha \sum_{i=1}^n |\theta_i|$$

The Lasso penalty term is an ℓ_1 -norm of the parameter vector, while the Ridge version uses an ℓ_2 -norm.

The Lasso term is not differentiable at $\theta_i = 0$, so the Gradient Descent algorithm needs to be adjusted properly using a *subgradient* method:

$$\theta^{[next\ step]} = \theta^{[previous\ step]} - \eta \nabla_{\theta} MSE \left(\theta^{[previous\ step]} \right) - a \odot sign(\theta)$$

where $sign(\theta)$ takes a value of -1 for those elements where the parameters are negative, 1 where the parameters are positive, and 0 where the parameters are strictly equal to 0.

Elastic Net Regression

The Elastic Net combines Ridge and Lasso regularization, where a hyperparameter determines the combination of shrinkage and feature selection.

The Elastic Net penalty is given by:

$$r\alpha \sum_{i=1}^n |\theta_i| + \frac{1-r}{2}\alpha \sum_{i=1}^n \theta_i^2$$

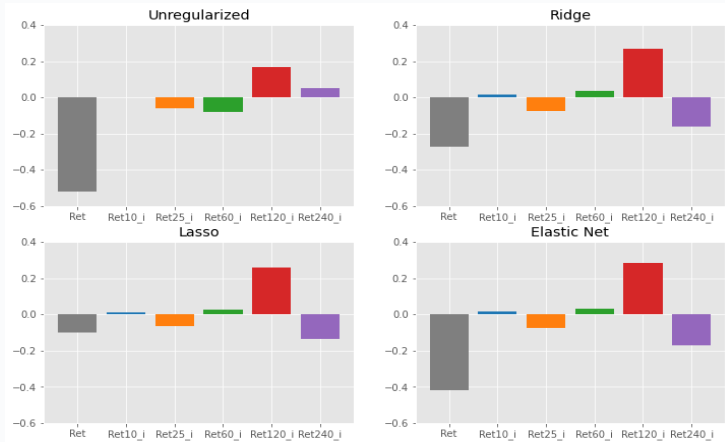
α determines the importance of regularization, r balances the mix of feature selection and shrinkage.

In an Elastic Net Regression with Gradient Descent, we would have, at least, three hyperparameters:

- ▶ Learning rate η .
- ▶ Overall regularization of the parameters α .
- ▶ Relative weight of Lasso vs. Ridge regularization r .

Below, we devise a method to obtain "good" values of those hyperparameters.

Time Series Momentum: Results



Hyperparameter tuning: Cross-validation

We may opt for a manual, *brute force* approach to check the best values for the hyperparameters. This would be quite inefficient because we may not have time to explore many combinations.

A more efficient technique exploits cross-validation: **k-fold cross-validation**.

In this method, the training set is split into k complementary subsets or “folds.” For each fold, the model is trained against the remaining folds using different hyperparameters, where the fold is used as a validation set.

Once the model type and hyperparameters have been selected, a final model is trained using these hyperparameters on the full training set, and the generalized error is measured on the test set.

Repeated k-fold cross-validation: We repeat the k-fold cross-validation process multiple times to avoid the noisiness from a small value of k .

Hyperparameter tuning: k-fold cross-validation illustration



Note: In all plots, the x-axis represents the predicted return from the models. The y-axis represents the actual return.

Hyperparameter tuning: Other tuners

The grid search approach is fine when you are exploring relatively few combinations, but it may get computationally costly when the hyperparameter search space is large.

Other alternatives for hyperparameter tuning:

- ▶ **Randomized search:** it evaluates random combinations of hyperparameters by selecting a random value for each hyperparameter at every iteration. Explore different values of the hyperparameters at each run, instead of pre-determined ones.
- ▶ **Ensemble methods:** allows for combining several predictors from different trained models to reach an even better predictor, combining the models that perform best.

Summary of Lesson 2

In Lesson 2, we have looked at:

- ▶ Methods to regularize ML models
- ▶ Methods to fine-tune the hyperparameters

⇒ **References for this lesson:**

Géron, Aurélien. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 2nd ed., O'Reilly Media, Inc., 2019.

TO DO NEXT: Now, please go to the associated Jupyter Notebook for this lesson to further explore the concepts covered in these slides and the application for our time series Momentum strategy.

In the next lesson, we will review the main concepts of ML classification problems.

Module 1: Lesson 3

Introduction to Machine Learning



Outline

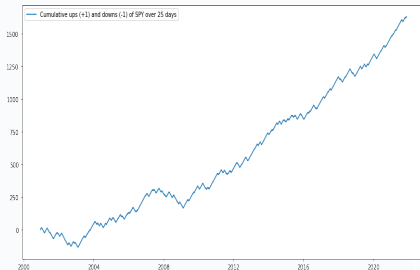
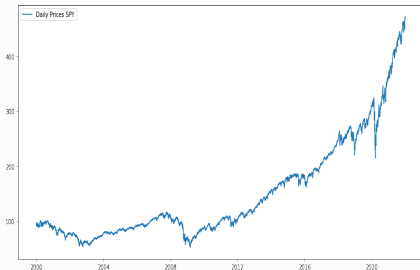
- ▶ Classification models and scaling
- ▶ Logistic Regression
- ▶ Performance of classification models

Classification models

Classification problems represent the most frequent tasks encountered in supervised ML applications. We want to predict whether an instance belongs to one class among an array of classes.

We can easily transform our return-prediction model developed in Lessons 1 and 2 into a classification problem.

That is, we may be interested in just predicting whether the stock market goes up or down, rather than a precise return. We can design simple short/long strategies based on our predictive model.



Scaling of input features

Feature scaling is one of the most important tasks to perform before we train an ML model, and this is not specific to classification problems.

Machine learning algorithms perform badly when the input features have very different scales, which tends to slow down and worsen the performance of the optimization algorithms (e.g., gradient descent).

Min-max scaling (or normalization): Values are shifted and re-scaled so that they end up ranging from 0 to 1:

$$x_j^{(i)} \leftarrow \frac{x_j^{(i)} - \min_j}{\max_j - \min_j}$$

Standardization: subtracts the mean value and divides by the variance so that the resulting distribution has zero mean and unit variance. It does not constrain values, which may be a problem in some algorithms.

Whitening: Using unsupervised ML methods, we create new de-correlated features, each of which is scaled to unit variance.

- **Important**: We first only scale the training set (using its min, max, mean...), which can then be applied to the validation and test sets.

Logistic Regression: Main elements

Logistic Regression is a popular approach in ML to deal with binary classification problems.

The model estimates the probability that an instance belongs to a particular class: For instance, what is the probability that the market will go up?

- In our training sample, we have instances with labels y that denote if the market has gone up $y = 1$ or gone down $y = 0$ and some input features X .

If the estimated probability is above 50%, the model predicts that the instance belongs to the positive class, $\hat{y} = 1$; otherwise, it belongs to the negative class, $\hat{y} = 0$.

To obtain the estimated probabilities for an instance i , the Logistic Regression model computes a score as a weighted sum of the input features (plus a bias term) and then outputs the logistic of this result:

$$\hat{p}^{(i)} = \frac{1}{1 + \exp(-x^{(i)}\theta)}$$

The function $\sigma(z) = \frac{1}{1 + \exp(-z)}$ is known as the sigmoid function. If we use the 50% threshold to determine the predicted class, the logistic regression then predicts 1 if $x^{(i)}\theta$ is positive and 0 otherwise.

Logistic Regression: Training

We train the parameter vector θ so that the model estimates high probabilities for positive instances ($y^{(i)} = 1$) and low probabilities for negative instances ($y^{(i)} = 0$).

This idea is captured by the following cost function:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

We have no closed form for the vector θ that maximizes the cost function above, but gradient descent is guaranteed to find the global minimum as long as the learning rate is small enough and we wait for enough time.

We can obtain the gradient from the derivatives from the cost function:

$$\frac{\partial}{\partial \theta_j} J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} - \sigma(x^{(i)}\theta) \right] x_j^{(i)}$$

We can regularize θ using the Ridge and Lasso penalties we described in Lesson 2.

Softmax Regression

We can generalize the binary classifier of the Logistic Regression model to multiple classes: This is called *Softmax Regression* or *Multinomial Logit Regression*.

The Softmax Regression model computes a score for each instance i and category c , $x^{(i)}\theta^{(c)}$, and estimates the probability of each class by using the softmax function.

- Notice that each category has a specific vector of parameters, $\theta^{(c)}$, with dimension equal to the number of input features, plus a bias term.

With C different categories, the estimated probability of category c for instance i is:

$$\hat{p}_c^{(i)} = \text{Softmax}_c \left(x^{(i)}, \Theta \right) = \frac{\exp \left(x^{(i)} \theta^{(c)} \right)}{\sum_{j=1}^C \exp \left(x^{(i)} \theta^{(j)} \right)}$$

where Θ collects all the parameters of the model. The model prediction $\hat{y}^{(i)}$ will be the category with the highest score $x^{(i)}\theta^{(c)}$.

Performance measures in classification

Evaluating a classifier is often significantly trickier than evaluating a regression model.

Accuracy: The frequency with which the classifier is successful at classifying the labels in the test sample.

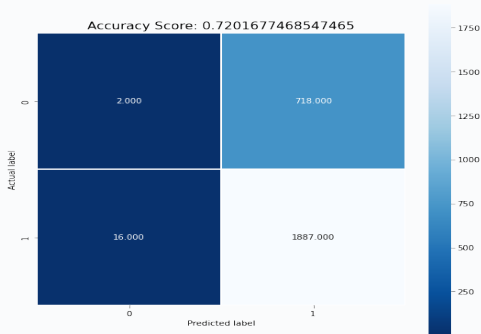
- ▶ Even a terrible classifier may be accurate. If the baseline probability of the positive class is 5%, a classifier that always predicts a negative class will mechanically achieve a 95% accuracy.

Using a Logistic Regression model, in our stock market prediction task, we find a 72% success rate in predicting whether the stock market is going up or down.

- ▶ The predictive power arises simply from the model predicting most of the time that the stock market is going up, which happens with roughly 72% probability.

Because accuracy is a somewhat misleading performance measure, we will describe other, more useful, metrics next.

Confusion Matrix



Confusion Matrix: After obtaining a set of predictions, we construct a matrix where each row represents the actual class of the instances, while each column represents the predicted classes. Each cell in the matrix then counts the number of times instances of class A are classified as B.

A perfect classifier would have nonzero values only on its main diagonal (top left to bottom right).

In the binary classifier case, we can represent the confusion matrix as:

$$\text{Confusion} = \begin{pmatrix} \text{True Negatives} & \text{False Positives} \\ \text{False Negatives} & \text{True Positives} \end{pmatrix}$$

Precision and recall

Precision: captures the proportion of accurate predictions over all the 1s that have been predicted by the model.

$$Precision = \frac{TP}{TP + FP}$$

where TP is the total number of true positives and FP is the number of false positives.

Recall: captures the proportion of accurate predictions over all the 1s that appear in the sample.

$$Recall = \frac{TP}{TP + FN}$$

Precision and recall are usually combined in the F_1 score $= 2 \frac{Precision \times Recall}{Precision + Recall}$

The F_1 score favors classifiers with similar precision and recall. Sometimes, we do not want that:

- If we develop a model to detect the probability of a large drop (crash) in the stock market, in which case the category is 1, we may be willing to sacrifice some accuracy, predict the crash when it does not take place, against some recall, do not predict the crash when the crash actually happens.

ROC & AUC

We cannot have, simultaneously, classifiers with high precision and high recall.

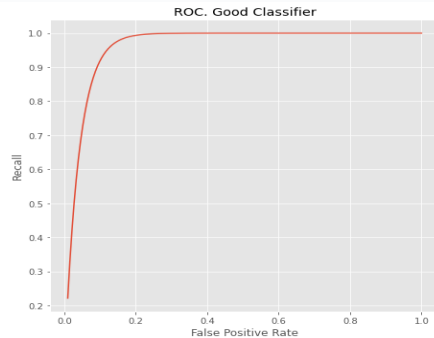
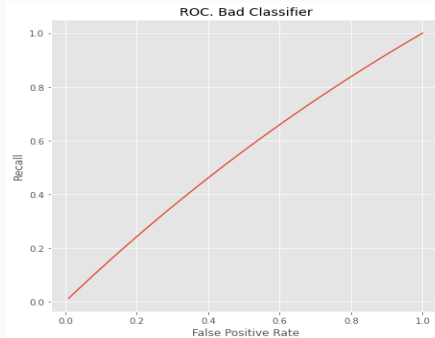
- ▶ To determine which class the model is predicting, we set a threshold value on the predicted probabilities to distinguish between a positive and a negative class, such as 0.5 in the logistic regression.
- ▶ If a classifier sets a high threshold to increase precision, it is going to increase the number of positive cases that are going to be predicted as negative, reducing recall.
- ▶ Otherwise, if a classifier sets a high bar to increase recall, it is going to be at the cost of predicting more negative cases as positives, reducing precision.

Still, a good classifier can increase its precision without a large decrease in recall, and vice versa.

To show this, we plot the Receiver Operating Characteristic (ROC) curve and compute the Area Under the Curve (AUC).

- ▶ The ROC curve plots the recall of the classifier against its rate of false positives for different thresholds for the estimated probabilities.
- ▶ A good classifier has a steep ROC that reaches a Recall close to 1 without generating a high rate of false positives. In that case, the AUC is close to 1.

ROC & AUC: Illustration



Summary of Lesson 3

In Lesson 3, we have looked at:

- ▶ Classification models in Supervised ML
- ▶ The Logistic Regression Model
- ▶ Performance measures for ML classification models

⇒ **References for this lesson:**

Géron, Aurélien. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 2nd ed., O'Reilly Media, Inc., 2019.

TO DO NEXT: Now, please go to the associated Jupyter Notebook for this lesson to obtain further insights on the workings of ML classification tasks and the performance of a stock market predictor.

In the next lesson, we will develop an application of ML classification models for a credit scoring problem.

Module 1: Lesson 4

Introduction to Machine Learning



Outline

- ▶ The setup and data preprocessing
- ▶ Setting up the logistic regression
- ▶ Performance: Confusion matrix and ROC
- ▶ Increasing the complexity of the model

Predicting the probability of default: The setup

The probability of default indicates the likelihood that a borrower will be unable to meet their debt payment obligations (either interest or principal).

- The higher the borrower's probability of default, the higher the interest rate charged by the lender, or the lower the amount the borrower would be eligible to obtain from the bank.

In this lesson, we exploit real-world information on default events from a bank in Israel to train a classification model to predict default cases of individuals.

We will exploit the informativeness that several variables may have in predicting the default behavior of individuals using a logistic regression model.

The model will be of use to banks or credit issuers to estimate the probability of default of an individual who approaches the bank to apply for a loan and who has certain characteristics.

Predicting the probability of default: Data

The data provides information about whether individuals default on their loans ($y = 1$) or not ($y = 0$) and several features (X) that may be informative about the expected default:

- ▶ Age.
- ▶ Education.
- ▶ Years with current employer.
- ▶ Years at current address.
- ▶ Household income.
- ▶ Debt to income ratio.
- ▶ Credit card debt.
- ▶ Other debt.

We transform categorical features (i.e., education) into sets of “dummy” variables that indicate if an individual belongs to a certain category:

- ▶ Use the “dummies” as input features in the ML problem, not the original categorical variable.

Predicting the probability of default: Preprocessing

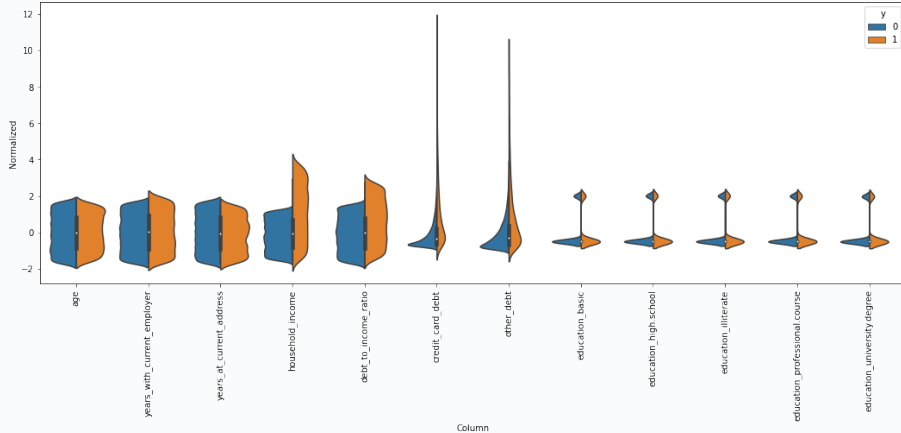
Before training an ML model, we should make sure that we have solid knowledge of the data we are working with.

Having some prior information about the data may very well improve our choices of ML algorithms or hyperparameters and lead to better prediction models.

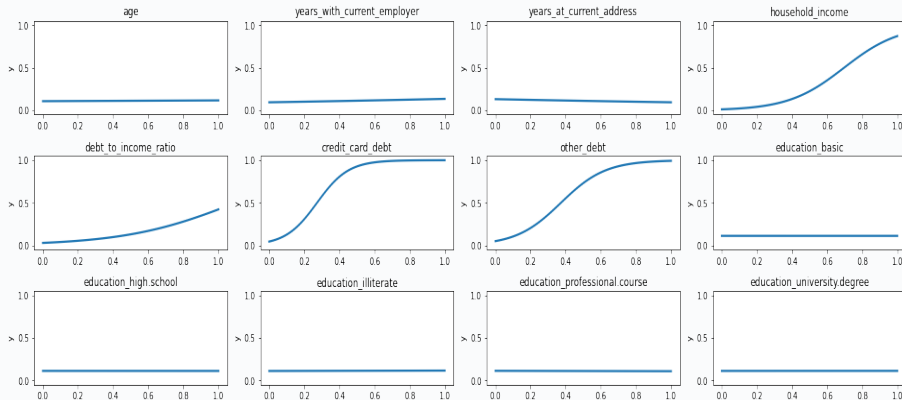
A checklist of steps that we must consider before training an ML model:

1. Analyze the descriptive statistics of the labels and input features.
2. Deal with missing data and outliers.
3. Select the most relevant features.
4. Rescale the features.

Predicting the probability of default: Summary statistics



Predicting the probability of default: Feature selection



Predicting the probability of default: Logistic Regression

We train a logistic regression model to achieve our classification task.

From the previous lesson, recall that the predicted probability of a given instance being a default case is:

$$\hat{p}^{(i)} = \frac{1}{1 + \exp(-x^{(i)}\theta)}$$

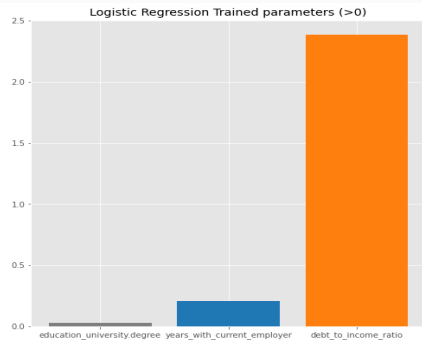
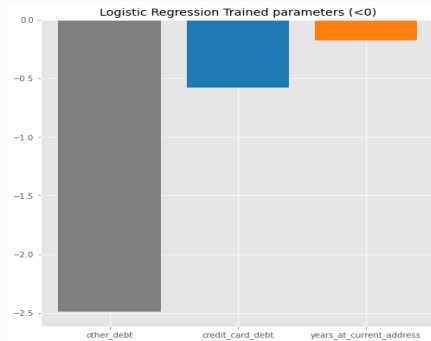
The objective of training is to minimize the cost function:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

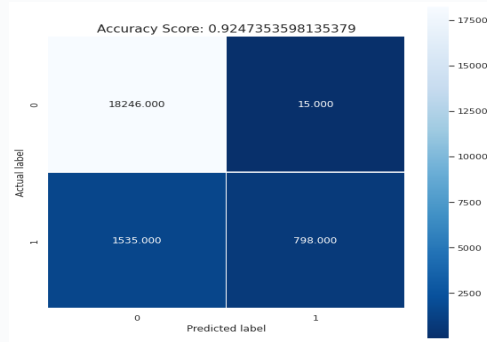
In Python's ScikitLearn, the Logistic Regression model includes by default a Ridge regularization with $\alpha = 1$.

$$\sum_{i=1}^n \theta_i^2$$

Predicting the probability of default: Trained parameters



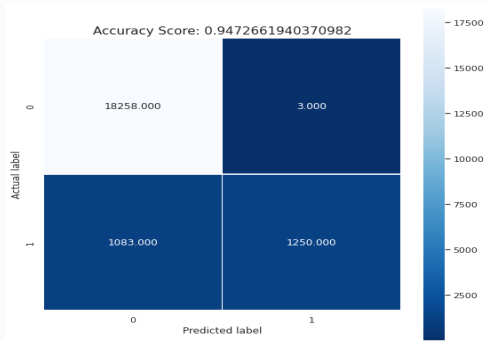
Predicting the probability of default: Confusion matrix



Predicting the probability of default: ROC



Predicting the probability of default: Powers of input features



Summary of Lesson 4

In Lesson 4, we have looked at:

- ▶ The preprocessing tasks that we need to perform before training an ML model
- ▶ Training a Logistic Regression model to predict the probability of default

⇒ **References for this lesson:**

Géron, Aurélien. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 2nd ed., O'Reilly Media, Inc., 2019.

TO DO NEXT: Now, please go to the associated Jupyter Notebook for this lesson to review the full implementation of the default prediction problem.

With this lesson, we end Module 1 as an introduction to Machine Learning. In later modules, you will learn about other useful and more advanced ML methods.

MACHINE LEARNING IN FINANCE

MODULE 1 | LESSON 1

INTRODUCTION TO MACHINE LEARNING

Reading Time	60 minutes
Prior Knowledge	Linear Algebra, Python
Keywords	Linear Regression, Classification, Machine learning

In this lesson, we describe the core concepts associated with machine learning. First, we define the boundaries among artificial intelligence, machine learning, and deep learning. Second, we introduce the main features of a machine learning problem in the context of the linear regression model and the time series predictability of stock prices.

1. Machine Learning

The terms machine (or deep) learning and artificial intelligence are frequently used in the same context or as synonyms. However, all these concepts refer to different things. Thus, we first need to define clearly what we are talking about when we mention all these terms. How do artificial intelligence, machine learning, and deep learning relate to each other?

1.1 Artificial Intelligence, Machine Learning and Deep Learning

Artificial intelligence (AI) is an academic discipline that studies the ability of any system that perceives its environment and that takes actions that maximize its chance of achieving its goal. AI represents the effort to automate intellectual tasks normally performed by humans.

As such, AI is a general field that encompasses machine learning and deep learning but that also includes many more approaches that don't involve any learning. For instance, early chess programs only involved hard-coded rules crafted by programmers and are different from machine learning. It turned out to be intractable to figure out explicit rules for solving complex problems, such as image classification or speech recognition. A new approach arose to solve these problems: machine learning.

Machine learning (ML) is often categorized as a subfield of AI that deals with computer programs that allow computers to learn from data. In classical programming, humans

input rules (a program) and data to be processed according to these rules, and the combination of both yields answers. With ML, humans input data as well as the answers expected from the data, while the program dictates the rules.

Fundamentally, ML involves building mathematical models to help understand data. Computers start "learning" when we insert tunable parameters in the programs that can be adapted to observed data. Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data.

A more precise definition of what requirements a ML program should satisfy is the following:

A computer program is said to learn from experience with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience.

An ML system is trained rather than explicitly programmed. It is presented with many examples relevant to a task, and it finds statistical structure in these examples that eventually allows the system to come up with rules for automating the task.

For instance, the spam filter of your email service is an ML program that learns to detect spam given examples of both spam emails (e.g., flagged by users) and regular emails. The examples that the system uses to learn are called the training set. The task is to flag spam for new emails, the experience is the training data, and the performance measure is, for example, the proportion of correctly classified emails.

Although ML only started to grow in the 1990s, it has quickly become the most popular and most successful subfield of AI, a trend driven by the availability of faster hardware and larger datasets. While ML is tightly related to mathematical statistics, it exhibits comparatively little mathematical theory, and it is engineering-oriented. It is a hands-on discipline in which ideas are most often proven empirically instead of theoretically.

Deep learning (DL) is a specific subfield of ML: A strategy for learning representations from data that puts an emphasis on learning successive layers of increasingly meaningful representations. Modern DL often involves dozens or even hundreds of successive layers of representations, which are all learned automatically from exposure to training data. Meanwhile, other approaches to ML tend to focus on learning only one or two layers of representations of the data; hence, they are sometimes called "shallow learning."

In DL, these layered representations are (almost always) learned via models called neural networks, structured in layers stacked on top of each other. The term neural network is a reference to neurobiology but, although some of the central concepts in Deep Learning were developed in part by drawing inspiration from our understanding of the brain, DL models are not models of the brain. DL is a mathematical framework for learning representations from data.

1.2 Supervised vs Unsupervised Learning

At the most fundamental level, ML can be categorized into two main types: supervised learning and unsupervised learning.

Supervised learning involves modeling the relationship between measured features of the training data and some label associated with the data. Once the model is trained and determined, it can be applied to other data with unknown labels. Supervised learning is further subdivided into classification tasks and regression tasks: in classification, the labels are discrete categories, while in regression, the labels are continuous quantities.

Unsupervised learning involves modeling the features of a dataset without reference to any label and is often described as "letting the dataset speak for itself." These models include tasks such as clustering and dimensionality reduction.

In this module, we are going to focus on basic supervised ML techniques. We cover unsupervised methods in later modules.

1.3 Elements of a Supervised Machine Learning Model

In supervised learning, the data is represented by some labels associated with certain features. The model is trained, and then it can make predictions for the labels of data that have been unused in training. In this way, the model learns from input features and targets (or labels).

- Features matrix (X): Composed of instances (rows) that have some input features (columns).
- Labels (also targets or output, y): A vector with the information we want the model to predict. For each instance in X there is a value, a label, in y .

The model can be represented as a function $g(\cdot)$ that takes the X matrix as input and tries to predict values as close as possible to the y targets. The computation developed to obtain the $g(\cdot)$ function is called training.

Types of supervised machine learning problems:

- Regression: the label is a number.
- Classification: the label is a category.

Supervised methods are about teaching the model by showing different examples, and the goal is to come up with a function that takes the features matrix X as a parameter and makes predictions as close as possible to the y labels.

2. Application of Supervised Learning: Time Series Momentum

We are going to study the basic elements of ML outlined above in the context of a simple financial application: time series momentum. Return momentum is considered as one of the most pervasive "anomalies" in Asset Pricing. We define return momentum as the positive autocorrelation of returns over different horizons. If the returns of an asset

displays momentum it means that we can predict with some accuracy its future returns based on past return information. This possibility challenges the basic definition of an efficient market, where historical information cannot be used to predict future returns.

There exist two main versions of portfolio strategies that exploit the existence of momentum in asset returns. The first strategy, which is a "cross-sectional" momentum strategy, focuses on buying stocks that have performed relatively well in the past and selling stocks that have performed relatively poorly.

The second strategy, a "time series" strategy, simply buys stocks that had a positive return in the past and sells stocks that had a negative return in the past, without a comparison across stocks. Our application in this lesson will be based on this notion of momentum trading by exploiting how much we can predict the evolution of an asset's price using just information about its past performance.

Variation in stock returns is difficult to pinpoint: Market-efficiency forces return variation to be dominated by unforecastable news that foreshadows expected returns. Regime switches, and non-stationarity in general diminish the effectiveness of return-prediction exercises. A further complication of the problem is the ambiguity regarding functional forms through which time series predictors are associated with future expected returns. All of these issues, among others, result in what is formally known as a low signal-to-noise ratio environment, which is in stark contrast to prediction problems in other fields. Thus, the application around which we are going to develop this module will illustrate the basic tools of ML and also the complicated task of developing useful predictive models of asset returns.

To exploit real world information, we are going to use the Yahoo Finance information that can be accessed with the library "yfinance." Our ML task in this application is to use a time series momentum strategy that exploits the predictability of SPY, an ETF that tracks the S&P500 index.

```
In [1]: import numpy as np
import yfinance as yf

# Getting historical market data from SPY (ETF) (SPY)
df = yf.download("SPY", start="2000-01-01", end="2022-01-01", auto_adjust=True)
df['Adj Close'] = df['Close'].values
```

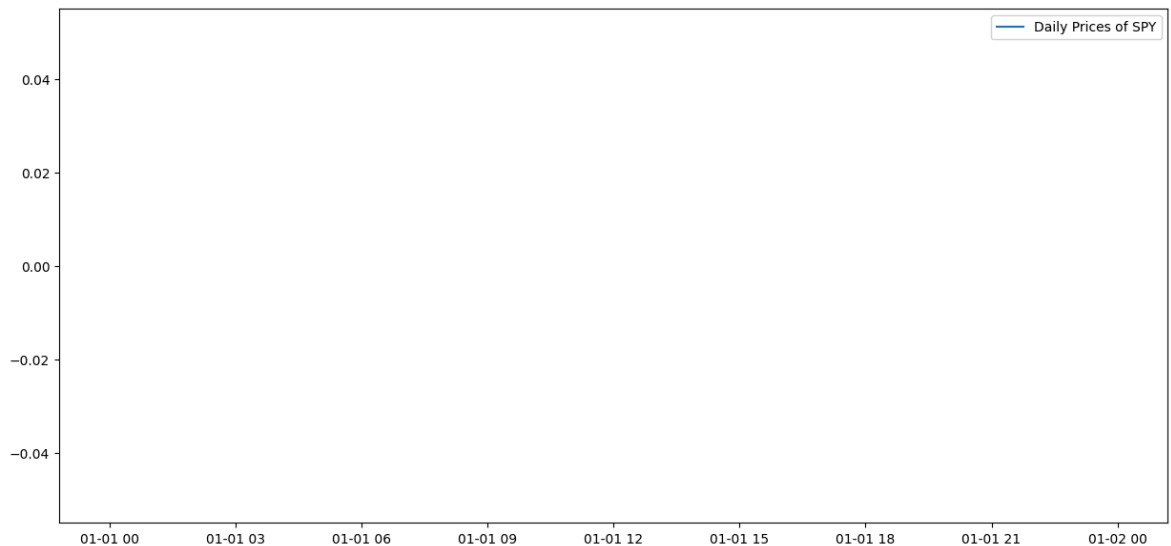
```
[*****100%*****] 1 of 1 completed
```

```
1 Failed download:
```

```
['SPY']: YFRateLimitError('Too Many Requests. Rate limited. Try after a while.')
```

```
In [2]: import matplotlib.pyplot as plt

plt.figure(figsize=(15, 7))
plt.plot(df.index, df["Adj Close"], label="Daily Prices of SPY")
plt.legend()
plt.show()
```



2.1 Features

We aim at forecasting the evolution of SPY 25 days ahead using as input features, $\$X\$$, the lags of SPY returns over 10, 25, 60, 120, and 240 days.

We rescale the information so that all the lags are expressed in daily returns.

```
In [3]: df["Ret"] = df["Adj Close"].pct_change()

name = "Ret"
df["Ret10_i"] = (
    df[name].rolling(10).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 10) - 1)
)
df["Ret25_i"] = (
    df[name].rolling(25).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 25) - 1)
)
df["Ret60_i"] = (
    df[name].rolling(60).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 60) - 1)
)
df["Ret120_i"] = (
    df[name].rolling(120).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 120) -
)
df["Ret240_i"] = (
    df[name].rolling(240).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 240) -
)

del df["Open"]
del df["Close"]
del df["High"]
del df["Low"]
del df["Volume"]
del df["Adj Close"]

df = df.dropna()
df.tail(10)
```

```
Out[3]: Price Ret Ret10_i Ret25_i Ret60_i Ret120_i Ret240_i
```

Ticker

Date

Thus, the features in this first example are given by:

- Ret10_i: Last 10-days compounded return
- Ret25_i: Last 25-days compounded return
- Ret60_i: Last 60-days compounded return
- Ret120_i: Last 120-days compounded return
- Ret240_i: Last 120-days compounded return
- Ret: Last day return

2.2 Target Variable

As mentioned above, our label (target) y is the vector of returns of SPY over the following 25 days.

```
In [4]: df["Ret25"] = df["Ret25_i"].shift(-25)
df = df.dropna()
df.tail(10)
```

```
Out[4]: Price Ret Ret10_i Ret25_i Ret60_i Ret120_i Ret240_i Ret25
```

Ticker

Date

3. Linear Regression

3.1 Our First Model for $g(\cdot)$: Linear Regression

One of the simplest (if not the simplest) models that we can train with machine learning is the linear regression model. Linear regression imposes $g(\cdot)$ to be a linear function of the inputs to predict the labels. A linear model makes a prediction by simply computing a sum of the input features, each multiplied by a parameter, plus a constant called the bias term (also "intercept"):
$$\widehat{y} = g(X) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$
 where

- $\widehat{y}$ is the predicted value for the label.
- n is the number of features.
- x_j is the j^{th} input feature.
- θ_k is the k^{th} model parameter, including the bias term θ_0 .

Suppose that our training sample contains m instances of labels and input features. Then, the instance i is represented by a label $y^{(i)}$ and a row vector of features $x^{(i)}$

$$\begin{aligned} x^{(i)} = & \left(x_0^{(i)}, x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)} \right) \\ \end{aligned} \quad \text{\$\$}$$

where the input x_0 takes a value of one in all instances, in association with the bias term.

To train the model, we need a criterion to determine how the model fits the training data. The most common performance measure of a regression model is the mean squared error (MSE). Therefore, if we stack the model parameters in a $n \times 1$ column vector θ , we need to find the values for each element of that vector that solves:
$$\underset{\theta}{\min} \frac{1}{m} \sum_{i=1}^m \left(y^{(i)} - x^{(i)} \theta \right)^2$$

The solution to this problem, $\hat{\theta}$, leads to a closed form expression, called the *Normal Equation*:

$$\hat{\theta} = (X^T X)^{-1} X^T y$$

where X^T is the transpose of the feature matrix X . The computations involved for such solution, particularly the inverse part, are extremely costly when we have information on a large number of features or too many instances to fit in the memory of the computer.

In machine learning applications, we usually opt for a generic optimization algorithm that is capable of finding optimal solutions to a wide range of problems: **Gradient Descent**. The idea behind this algorithm is to adjust gradually the parameters of the model in order to minimize a cost function, in our case the MSE. The method measures the local gradient of the cost function with respect to a parameter and tweaks the parameters in the direction of the descendent gradient. When the gradient is zero, we have reached a minimum.

An important element of gradient descent is the *learning rate* hyperparameter (more on hyperparameters later). The learning rate determines the size of the adjustment of the parameters after each evaluation of the gradient. A too low learning rate will lead to many iterations in the algorithm; a too high learning rate will lead to divergence in the algorithm.

One option to proceed is to fill θ with random values, *random initialization*, and then improve the cost function gradually until the algorithm converges to a minimum. Fortunately, the MSE cost function for a linear regression model happens to be a *convex function* that has one global minimum. Hence, Gradient Descent is guaranteed to approach arbitrarily close to the unique minimum, as long as the learning rate is not too high. Moreover, transforming the input features to have a similar scale among them will also help to increase the speed of convergence.

To implement gradient descent, we need to compute the gradient of the cost function with respect to each model parameter θ_k , that is, the *partial derivative*. When we use the MSE as a cost function, the partial derivative is given by:
$$\frac{\partial}{\partial \theta_j} \text{MSE}(\theta) = -\frac{2}{m} \sum_{i=1}^m \left(y^{(i)} - \right.$$

$$x^{(i)} \theta_j^{(i)}$$

$$x_0^{(i)} = 1$$
 for all i . If we stack all the partial derivatives in a column vector, we obtain the gradient $\nabla_{\theta} \text{MSE}(\theta)$, and each step in Gradient Descent leads to an update of the parameter vector given by:

$$\theta^{[\text{next step}]} = \theta^{[\text{previous step}]} - \eta \nabla_{\theta} \text{MSE}(\theta^{[\text{previous step}]})$$
 where $\eta > 0$ is the learning rate. We stop the algorithm when the gradient vector becomes tiny, below a tolerance ϵ . To find a good learning rate, we can perform a grid search and eliminate models with a too slow convergence.

The Gradient Descent algorithm introduced above is called *Batch Gradient Descent*; it uses the whole batch of training data at every step to update the parameters. This method can be slow on very large training sets. To reduce the computational cost of each step we can opt for *Stochastic Gradient Descent*, which computes the gradient at each step based on a single instance picked at random, or for *Mini-Batch Gradient Descent*, which computes the gradients at each step based on random sets of instances (called mini-batches).

With all these concepts in mind, let's train our linear model and determine its ability predict future returns in the training sample. First, we load some libraries needed in this notebook. The Scikit-Learn library will be useful in all the applications of this module.

```

In [5]: import matplotlib.pyplot as plt
        from sklearn import linear_model
        from sklearn.metrics import mean_squared_error, r2_score
  
```

Let's define our input matrix X and label vector y . We have 6 input features and 5,271 instances.

```

In [6]: Xdf, ydf = df.iloc[:, 0:-1], df.iloc[:, -1]
        X = Xdf.astype("float32")
        y = ydf.astype("float32")
        print(X.shape, y.shape)
  
```

(5271, 6) (5271,)

We now train the model to obtain the coefficients that minimize the MSE:

```

In [7]: reg = linear_model.LinearRegression().fit(X, y)
  
```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[7], line 1
----> 1 reg = linear_model.LinearRegression().fit(X, y)

File /usr/local/lib/python3.11/site-packages/sklearn/base.py:1389, in _fit_context.<locals>.decorator.<locals>.wrapper(estimator, *args, **kwargs)
    1382 estimator._validate_params()
    1384 with config_context(
    1385     skip_parameter_validation=(
    1386         prefer_skip_nested_validation or global_skip_validation
    1387     )
    1388 ):
-> 1389     return fit_method(estimator, *args, **kwargs)

File /usr/local/lib/python3.11/site-packages/sklearn/linear_model/_base.py:601, in LinearRegression.fit(self, X, y, sample_weight)
    597 n_jobs_ = self.n_jobs
    599 accept_sparse = False if self.positive else ["csr", "csc", "coo"]
--> 601 X, y = validate_data(
    602     self,
    603     X,
    604     y,
    605     accept_sparse=accept_sparse,
    606     y_numeric=True,
    607     multi_output=True,
    608     force_writeable=True,
    609 )
    611 has_sw = sample_weight is not None
    612 if has_sw:

File /usr/local/lib/python3.11/site-packages/sklearn/utils/validation.py:2961, in validate_data(_estimator, X, y, reset, validate_separately, skip_check_array, **check_params)
    2959     y = check_array(y, input_name="y", **check_y_params)
    2960     else:
-> 2961     X, y = check_X_y(X, y, **check_params)
    2962     out = X, y
    2964 if not no_val_X and check_params.get("ensure_2d", True):

File /usr/local/lib/python3.11/site-packages/sklearn/utils/validation.py:1370, in check_X_y(X, y, accept_sparse, accept_large_sparse, dtype, order, copy, force_writeable, force_all_finite, ensure_all_finite, ensure_2d, allow_nd, multi_output, ensure_min_samples, ensure_min_features, y_numeric, estimator)
    1364     raise ValueError(
    1365         f"{estimator_name} requires y to be passed, but the target y is None"
    1366     )
    1368 ensure_all_finite = _deprecate_force_all_finite(force_all_finite, ensure_all_finite)
-> 1370 X = check_array(
    1371     X,
    1372     accept_sparse=accept_sparse,
    1373     accept_large_sparse=accept_large_sparse,
    1374     dtype=dtype,
    1375     order=order,
    1376     copy=copy,
    1377     force_writeable=force_writeable,
    1378     ensure_all_finite=ensure_all_finite,
    1379     ensure_2d=ensure_2d,

```

```

1380     allow_nd=allow_nd,
1381     ensure_min_samples=ensure_min_samples,
1382     ensure_min_features=ensure_min_features,
1383     estimator=estimator,
1384     input_name="X",
1385 )
1387 y = _check_y(y, multi_output=multi_output, y_numeric=y_numeric, estimator
=estimator)
1389 check_consistent_length(X, y)

```

File /usr/local/lib/python3.11/site-packages/sklearn/utils/validation.py:1130, in check_array(array, accept_sparse, accept_large_sparse, dtype, order, copy, force_writeable, force_all_finite, ensure_all_finite, ensure_non_negative, ensure_2d, allow_nd, ensure_min_samples, ensure_min_features, estimator, input_name)

```

1128     n_samples = _num_samples(array)
1129     if n_samples < ensure_min_samples:
-> 1130         raise ValueError(
1131             "Found array with %d sample(s) (shape=%s) while a"
1132             " minimum of %d is required%s."
1133             % (n_samples, array.shape, ensure_min_samples, context)
1134         )
1136 if ensure_min_features > 0 and array.ndim == 2:
1137     n_features = array.shape[1]

```

ValueError: Found array with 0 sample(s) (shape=(0, 6)) while a minimum of 1 is required by LinearRegression.

```
In [ ]: reg.coef_
```

```
In [ ]: reg.intercept_
```

Notice that the signs of the coefficients vary, which implies that there is a negative and positive association between the returns of SPY in 25 days and the returns over different past horizons. These are the typical features of a mean-reverting process.

To understand how well the model fits the training data, we can directly use the MSE of the model. However, the MSE is an absolute measure, so we need some benchmark to evaluate the performance of the model. A useful alternative is to compute the coefficient of determination, or R^2 , defined as:

$$R^2 = 1 - \frac{\sum_{i=1}^m \left(y^{(i)} - \widehat{x^{(i)}}_{\theta} \right)^2}{\sum_{i=1}^m \left(y^{(i)} - \overline{y} \right)^2}$$

where $\overline{y}$ is the average of the labels in the training sample.

The R^2 of the linear regression is a measure of a goodness of fit that indicates the proportion of the variation of the labels that the model can explain. We can also understand the R^2 as the prediction gains made by the model against an alternative (benchmark) prediction model that just uses the sample average to predict the labels. In the linear regression setup, if the model includes a bias term, the model that includes one or several input features will always perform at least marginally better than the average prediction in the training sample. In other setups, like out of the training sample or with other models, we can perform arbitrarily worse than the average at making predictions, in which case the R^2 can be negative.

Unfortunately, we show below that the linear model demonstrates poor performance in predicting returns in the training sample. In the scatter diagram, we show below, we plot the relationship between the predicted return (horizontal axis) and the actual return (vertical axis). In a model that displays a perfect fit, R^2 is one, and all the points would lie on the black line that represents the 45-degree diagonal, that is we would have $\widehat{y}=y$. The model is far from achieving that.

```
In [ ]: reg.score(X, y) # R-squared
```

```
In [ ]: # Make predictions
y_pred = reg.predict(X)

# The coefficients
print("Coefficients: \n", reg.coef_)
# The mean squared error
print("Mean squared error: %.5f" % mean_squared_error(y, y_pred))
# The coefficient of determination: 1 is perfect prediction
print(
    "Coefficient of determination (R2): %.5f" % r2_score(y, y_pred)
) # Another way to compute the R-squared
```

```
In [ ]: # Plot outputs
plt.scatter(y_pred, y)
plt.plot([-1, 1], [-1, 1], color="black", linewidth=2)
plt.show()
```

3.2 Out-of-Sample vs In-Sample Performance

In supervised learning, we are interested in training models that are useful to predict the realizations of the labels y of instances whose input features are not used while training the model. To test the predictive ability of the model, we split the data into two samples, a training sample from where we train the model, and a test sample that allows us to evaluate the predictive, or out-of-sample, ability. The out-of-sample error tells us how well the model performs with instances it has never seen before.

In our particular application, we are going to split the sample in half into training and test samples.

```
In [ ]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=int(len(y) * 0.5), shuffle=False
)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

Let's train the model with this new training data.

```
In [ ]: reg = linear_model.LinearRegression().fit(X_train, y_train)
```

```
In [ ]: # Make predictions in the training sample
y_pred = reg.predict(X_train)

# The coefficients
```

```

print("Coefficients: \n", reg.coef_)
# The mean squared error
print("Mean squared error: %.5f" % mean_squared_error(y_train, y_pred))
# The coefficient of determination: 1 is perfect prediction
print("Coefficient of determination (R2): %.5f" % r2_score(y_train, y_pred))

```

```

In [ ]: # Make predictions in the test sample
y_pred_test = reg.predict(X_test)

# The coefficients
print("Coefficients: \n", reg.coef_)
# The mean squared error
print(
    "Mean squared error (out-of-sample): %.5f" % mean_squared_error(y_test, y_pr
)
# The coefficient of determination: 1 is perfect prediction
print(
    "Coefficient of determination (R2 out-of-sample): %.5f"
    % r2_score(y_test, y_pred_test)
)

```

```

In [ ]: # Plot outputs
plt.scatter(y_pred, y_train, label="Train")
plt.scatter(y_pred_test, y_test, label="Test")
plt.plot([-0.5, 0.5], [-1, 1], color="black", linewidth=2)
legend = plt.legend(loc="upper left", shadow=True)
plt.show()

```

Notice that using a different training sample leads to different parameters from the trained model that exploit the full data. However, the R^2 is still low and the scatterplot above illustrates the poor fit. This model then features an *underfitting* problem. The model is too simple to learn the underlying structure of the data. Linear models are prone to underfit, reality is more complex than the model, and predictions are bound to be inaccurate even on the training sample. If we wish to solve the underfitting of a model, we just need to opt for more complex and flexible models or introduce new input features that may better predict the labels.

Moreover, if we observe the out-of-sample (i.e., test) R^2 above, we obtain a negative number. This means that using a prediction that just exploits the sample average of the returns over the test sample would perform better than the trained model. Usually, this low R^2 indicates that we are using a oversimplified (underfit) model. You will see how to improve it, using more advanced models, in the next modules.

Why is the R^2 negative? Notice that we train our model to minimize the MSE in the training sample, ignoring the information from the test sample. Indeed, the predictive ability of the model in the training sample has increased relative to the model that uses the whole information for training. The trained model may perform well to match the data in the training sample, but it may be terrible when it faces new information. The MSE of the trained model in the test sample may be much worse than the MSE generated by a prediction based on the average. This boils down to the problem of *overfitting* in machine learning, which we describe next.

Would our predictive model improve by adding further complexity to the model? To further illustrate the overfitting problem, we are going to increase the complexity in the Linear Regression simplest way. This means adding powers of each feature as new features and then train a linear model on this extended set of features. This technique is called Polynomial Regression.

```
In [ ]: # Polynomial regression
Xpoly = X
for pp in range(2, 5):
    Xpoly = np.concatenate((Xpoly, np.power(X, pp)), axis=1)

X_train, X_test, y_train, y_test = train_test_split(
    Xpoly, y, test_size=int(len(y) * 0.5), shuffle=False
)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)

reg = linear_model.LinearRegression().fit(X_train, y_train)

# Make predictions in the training sample
y_pred = reg.predict(X_train)

# The coefficients
print("Coefficients: \n", reg.coef_)
# The mean squared error
print("Mean squared error: %.5f" % mean_squared_error(y_train, y_pred))
# The coefficient of determination: 1 is perfect prediction
print("Coefficient of determination (R2): %.5f" % r2_score(y_train, y_pred))

# Make predictions in the test sample
y_pred_test = reg.predict(X_test)

# The coefficients
print("Coefficients: \n", reg.coef_)
# The mean squared error
print(
    "Mean squared error (out-of-sample): %.5f" % mean_squared_error(y_test, y_pr
)
# The coefficient of determination: 1 is perfect prediction
print(
    "Coefficient of determination (R2 out-of-sample): %.5f"
    % r2_score(y_test, y_pred_test)
)

# Plot outputs
plt.scatter(y_pred, y_train, label="Train")
plt.scatter(y_pred_test, y_test, label="Test")
plt.plot([-1, 1], [-1, 1], color="black", linewidth=2)
legend = plt.legend(loc="upper left", shadow=True)
plt.show()
```

As you can see in the results above, the prediction performance of the model in the training sample improves and the underfitting decreases, while the out-of-sample performance declines considerably, relative to the basic linear regression model. This is an illustrative example of overfitting. We increase the complexity of the model, but this complex model seems to learn from noise, which damages its out-of-sample performance.

3.3 Overfitting

Generalizing from a few observations is usually a trap humans fall into, but machines can also make the same mistake if we are not careful. In Machine Learning, this is called overfitting: the model performs well on training data but does not generalize well when we feed new data into the trained model.

A model's generalization error can be expressed as the sum of three very different errors:

- Bias. This part of the generalization error is due to wrong assumptions, such as assuming that the data is linear when it is actually quadratic. A high-bias model is most likely to underfit the training data.
- Variance. This part is due to the model's excessive sensitivity to small variations in the training data. A model with many degrees of freedom (such as a high-degree polynomial model) is likely to have high variance, and thus to overfit the training data.
- Noise or irreducible error. This part is due to the noisiness of the data itself. The only way to reduce this part of the error is to clean up the data (e.g., fix the data sources or detect and remove outliers).

Increasing a model's complexity will typically increase its variance and reduce its bias. Conversely, reducing a model's complexity increases its bias and reduces its variance. This is why we face a bias-variance trade-off in machine learning.

Sometimes, we do not win with more powerful (i.e., less biased) models when working with limited training data, because of the higher variance of these models. For example, if we change the training data for a model, we are likely to learn a completely different set of parameters (due to its intrinsic noise). The new model is likely to yield a completely different prediction on the same test instance as compared to the predictions using the first training data set. This type of variation in the prediction of the same test instance using different training data sets is a manifestation of model variance, which also adds to the error of the model. Both predictions of the same test instance could not possibly be correct.

If our training set is too small, or too noisy, complex models are probably going to detect patterns from noise. This noise will not generalize to new instances and our model will fail. Some options to reduce the overfitting problem consist of gathering more training data to reduce noise or cleaning the data to fix errors or remove outliers.

Another strategy to reduce overfitting in a complex model is to constrain or *regularize* the parameters. For instance, in the linear regression model above we can restrict certain model parameters to be relatively small, reducing the degrees of freedom to adapt the model. Regularization allows us to find a balance between fitting the data perfectly and keeping the model simple enough to ensure that it will generalize well. In the next lesson, we will describe the most relevant regularization methods.

The amount of regularization to apply during learning can be controlled by a *hyperparameter*, which is a parameter of a learning algorithm, but that does not belong to the prediction model. As such, hyperparameters are fixed during training. Hyperparameter-tuning is an important part of Machine Learning systems.

The choice of hyperparameters allows to discriminate across models to find the one that best performs out-of-sample. Still, a model that performs well with a particular choice of hyperparameters may display a poor performance once we feed the model with new instances. This arises because we chose the hyperparameters to generate the best fit for the test set.

A common solution to this problem is to split our data further into a *validation set*. Then, the choice of models will be done after fitting different models with different hyperparameters in the training set, determine the best model, and hyperparameter based on their performance in the validation set, and then run the model in the test set to have a sense of the generalization error of the model.

An efficient way to regularize learning algorithms, such as Gradient Descent, is to stop training as long as the validation error reaches a minimum. This is called *early stopping*. This technique stops training when the algorithm starts to overfit the training sample. In that case, we use Mini-Batch or Stochastic Gradient Descent, which exploit random sets of data, and we can use an early stopping rule that stops training if the validation error has been above the minimum for some time. Then, we can roll back the parameters of the model to the point where the minimum error in validation was found.

4. Conclusion

In this first lesson, we have covered the introductory concepts of machine learning. In the next lesson, we will cover the main techniques for parameter regularization, cross-validation, and hyperparameter tuning.

See you there!

MACHINE LEARNING IN FINANCE

MODULE 1 | LESSON 2

SUPERVISED MODELS: REGRESSION AND HYPERPARAMETERS

Reading Time 60 minutes

|Prior Knowledge | Linear regression

|Keywords | Penalized regressions, linear models, hyperparameters |

In this lesson, we will introduce in a linear regression framework the main techniques for parameter regularization, cross-validation, and hyperparameter tuning.

1. Linear Models and Overfitting

In the linear regression context, a good way to reduce the overfitting of a model is to regularize (constrain) the parameters, sometimes also called "weights." Regularization reduces the degrees of freedom of the model to fit the training data.

In general form, let $J(\theta)$ denote the cost function, during training, of a model with vector of parameters θ . Then, regularization is achieved by adding a penalty term to the MSE of the model, so that the cost function from linear regression becomes:
$$J(\theta) = \text{MSE}(\theta) + \text{Penalty}(\theta)$$

We next describe three methods--Ridge, Lasso, and Elastic Net--which specify penalty, or regularization, terms to constrain the weights. One important aspect is that, in any method, the penalty term should only be added to the cost function during training. In the validation or test samples, we are only interested in evaluating the unregularized performance of the model.

1.1 Ridge

The Ridge method is called a *shrinkage* regularization technique that aims at keeping the smallest values possible for the parameters of the model. To achieve this, the regularization term is of the form
$$\alpha \sum_{i=1}^n \theta_i^2$$
 The hyperparameter α determines how much we wish to regularize the model. If $\alpha=0$, we are trivially not regularizing the model while, if α is very large, then all parameters will be close to zero and our fit of the data will

be a flat line through the average of the labels y in the training sample (Notice that the regularization term above does not include the bias term θ_0).

The cost function for a linear regression with a ridge penalty (Ridge Regression) is then given by

$$J(\theta) = \text{MSE}(\theta) + \frac{1}{2} \alpha \sum_{i=1}^n \theta_i^2$$

Thus, if we perform gradient descent, the update of the parameters will be given by:

$$\theta^{\{\text{next step}\}} = \theta^{\{\text{previous step}\}} - \eta \nabla_{\theta} \text{MSE}(\theta^{\{\text{previous step}\}}) - \alpha \odot \theta^{\{\text{previous step}\}}$$

where α is a column vector of n elements all of which have the value α , except the first term that has a value of 0, in association with the term θ_0 not being regularized, and $\odot$ refers to element-by-element multiplication. Finally, notice that the $1/2$ multiplying α and the parameters in the penalty function is just to get a cleaner derivative once we compute the gradient.

1.2 Lasso

The Lasso (Least Absolute Shrinkage and Selection Operator) penalty is another regularization method that tends to eliminate the weights of the least important features, setting them to zero. The regularization term is of the form:

$$\alpha \sum_{i=1}^n |\theta_i|$$

Notice that this term is an ℓ_1 norm, while the Ridge version uses an ℓ_2 norm of the parameter vector. The hyperparameter α again determines the overall importance of the regularization of parameters. The Lasso Regression cost function then reads as:

$$J(\theta) = \text{MSE}(\theta) + \alpha \sum_{i=1}^n |\theta_i|$$

The Lasso Regression performs a feature selection and outputs a sparse model with few non-zero parameters. Because the Lasso term is not differentiable at $\theta_i=0$, the gradient descent algorithm needs to be adjusted properly using a *subgradient* method:

$$\theta^{\{\text{next step}\}} = \theta^{\{\text{previous step}\}} - \eta \nabla_{\theta} \text{MSE}(\theta^{\{\text{previous step}\}}) - \alpha \odot \begin{pmatrix} 0 \\ \text{sign}(\theta_1) \\ \vdots \\ \text{sign}(\theta_n) \end{pmatrix}$$

where

$$\text{sign}(\theta_i) = \begin{cases} -1 & \text{if } \theta_i < 0 \\ 0 & \text{if } \theta_i = 0 \\ +1 & \text{if } \theta_i > 0 \end{cases}$$

1.3 Elastic Net

The Elastic Net combines Ridge and Lasso regularization in a single optimization, where a hyperparameter determines the relative combination of shrinkage and feature selection when training the model. The Elastic Net cost function is given by:

$$J(\theta) = \text{MSE}(\theta) + r \alpha \sum_{i=1}^n |\theta_i| + \frac{1}{2} \alpha \sum_{i=1}^n \theta_i^2$$

The hyperparameter α determines the overall importance of the regularization of parameters, while the hyperparameter λ balances the mix between feature selection and parameter shrinkage.

Which of the three methods is preferable? Ridge is a good default, but we guess that only a few features are actually useful. You should opt for Lasso or Elastic Net because they remove the features that are useless. In general, a plain Lasso may behave erratically when the number of features is greater than the number of training instances or when several features are strongly correlated. Thus, the Elastic Net is a good choice.

Notice that in an Elastic Net Regression with Gradient Descent optimization, we would have, at least, three hyperparameters: The learning rate η , the degree of overall regularization of the parameters α , and the relative weight of Lasso vs. Ridge regularization λ . Next, we devise methods to obtain "good" values of those hyperparameters.

1.4 Elastic Net Regression and Times Series Momentum

To illustrate the workings of the Elastic Net Regression, we are going to return to our task of predicting stock returns based on past information.

We first upload the necessary packages and define the input feature matrix X and labels y as in the previous lesson. We also divide the sample into a training set and a test set (we will introduce the validation set below).

```
In [ ]: import numpy as np
import pandas as pd
import yfinance as yf

# Getting historical market data from SPY (ETF) (SPY)
df = yf.download("SPY", start="2000-01-01", end="2022-01-01")

In [ ]: df["Ret"] = df["Adj Close"].pct_change()

name = "Ret"
df["Ret10_i"] = (
    df[name].rolling(10).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 10) - 1)
)
df["Ret25_i"] = (
    df[name].rolling(25).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 25) - 1)
)
df["Ret60_i"] = (
    df[name].rolling(60).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 60) - 1)
)
df["Ret120_i"] = (
    df[name].rolling(120).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 120) - 1)
)
df["Ret240_i"] = (
    df[name].rolling(240).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 240) - 1)
)

del df["Open"]
del df["Close"]
```

```
del df["High"]
del df["Low"]
del df["Volume"]
del df["Adj Close"]

df = df.dropna()
df.tail(10)
```

```
In [ ]: df["Ret25"] = df["Ret25_i"].shift(-25)
df = df.dropna()
df.tail(10)
```

```
In [ ]: X, y = df.iloc[:, 0:-1], df.iloc[:, -1]
print(X.shape, y.shape)
```

```
In [ ]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=int(len(y) * 0.5), shuffle=False
)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

We first study separately the performance of ridge regression and lasso regression on their own. Recall that ridge regression forces the learning algorithm to not only fit the data but also keep the model parameters θ as small as possible.

In the code below, we keep the regularization parameter α fixed to 0.7.

```
In [ ]: import matplotlib.pyplot as plt

# import ridge regression from sklearn library
from sklearn.linear_model import Ridge
```

```
In [ ]: ridgeR = Ridge(alpha=0.7)
ridgeR.fit(X_train, y_train)
y_pred_ridge = ridgeR.predict(X_train)
y_pred_test_ridge = ridgeR.predict(X_test)

# calculate mean squared error
mean_squared_error = np.mean((y_pred_test_ridge - y_test) ** 2)
print("Mean squared error on test set", mean_squared_error)
```

```
In [ ]: # get ridge coefficient and print them
ridge_coefficient = pd.DataFrame()
ridge_coefficient["Columns"] = X_train.columns
ridge_coefficient["Coefficient Estimate"] = pd.Series(ridgeR.coef_)
print(ridge_coefficient)
```

```
In [ ]: # plotting the coefficient score
fig, ax = plt.subplots(figsize=(15, 7))

color = [
    "tab:gray",
    "tab:blue",
    "tab:orange",
    "tab:green",
    "tab:red",
```

```

        "tab:purple",
        "tab:brown",
        "tab:pink",
        "tab:gray",
        "tab:olive",
        "tab:cyan",
        "tab:orange",
        "tab:green",
        "tab:blue",
        "tab:olive",
    ]

ax.bar(
    ridge_coefficient["Columns"], ridge_coefficient["Coefficient Estimate"], col
)

plt.style.use("ggplot")
plt.show()

```

Check how the model parameters have changed from training the model with the ridge regularization relative to the simple linear regression model. The parameter with the highest absolute value in the trained linear regression, the one associated with the most recent daily return, is adjusted downwards (in absolute terms). In contrast, the parameters on the most distant lags increase their relative importance. However, ridge regularization seems to add little in terms of prediction ability relative to linear regression (compared with the results we obtained in Lesson 1).

Let's see how we can implement Lasso regularization in our application.

```

In [ ]: # import Lasso regression from sklearn library
        from sklearn.linear_model import Lasso

        # Train the model
        lasso = Lasso(alpha=0.0001)
        lasso.fit(X_train, y_train)
        y_pred_lasso = lasso.predict(X_train)
        y_pred_test_lasso = lasso.predict(X_test)

        # Calculate Mean Squared Error
        mean_squared_error = np.mean((y_pred_test_lasso - y_test) ** 2)
        print("Mean squared error on test set", mean_squared_error)

```

```

In [ ]: lasso_coeff = pd.DataFrame()
        lasso_coeff["Columns"] = X_train.columns
        lasso_coeff["Coefficient Estimate"] = pd.Series(lasso.coef_)

        print(lasso_coeff)

```

Notice how Lasso regularization shrinks further the magnitude of the parameters relative to ridge regression, but none reaches a value of zero. Thus, none of the input features of the model are considered as irrelevant by the algorithm. Still, the trained model does not fare better than the Ridge version.

Let's try now with the Elastic Net Regression model that combines both regularization features. Notice that the parameter "l1_ratio" in the code below determines the relative

importance of lasso against ridge regularization in the model.

We can see from the out-of-sample MSE that the combination of both regularization methods contributes little to our prediction problem, as we would expect from observing their separate performance.

```
In [ ]: # import model
        from sklearn.linear_model import ElasticNet

        # Train the model
        e_net = ElasticNet(alpha=0.0001, l1_ratio=0.1)
        e_net.fit(X_train, y_train)

        # calculate the prediction and mean square error
        y_pred_elastic = e_net.predict(X_test)
        mean_squared_error = np.mean((y_pred_elastic - y_test) ** 2)
        print("Mean Squared Error on test set", mean_squared_error)
```

```
In [ ]: e_net_coeff = pd.DataFrame()
        e_net_coeff["Columns"] = X_train.columns
        e_net_coeff["Coefficient Estimate"] = pd.Series(e_net.coef_)
        e_net_coeff
```

2. Hyperparameter Tuning

Let's assume that you now have a shortlist of promising models. You now need to fine-tune them. Let's look at a few ways you can do that.

2.1 Cross-Validation: GridSearchCV

One way to check what the best values are for the hyperparameters would be to manually check the performance of the models with different hyperparameters, but this would be quite inefficient because we may not have time to explore many combinations.

A more efficient technique exploits cross-validation: the training set is split into complementary subsets or "folds." For each fold, the model is trained against the remaining folds using different hyperparameters, where the fold is used as a validation set. This is called *k-fold cross-validation*. Once the model type and hyperparameters have been selected, a final model is trained using these hyperparameters on the full training set, and the generalized error is measured on the test set.

Scikit-Learn's GridSearchCV will do the fine-tuning and the cross validation. All you need to do is tell it which hyperparameters you want it to experiment with and what values to try out and it will evaluate all the possible combinations of hyperparameter values, using cross-validation.

2.2 Other Tuners

The grid search approach is fine when you are exploring relatively few combinations, but when the hyperparameter search space is large, it is often preferable to use *Randomized*

Search through RandomizedSearchCV instead. This class can be used in much the same way as the GridSearchCV class, but instead of trying out all possible combinations, it evaluates a given number of random combinations of hyperparameters by selecting a random value for each hyperparameter at every iteration. An advantage over grid search is that it will explore different values of the hyperparameters at each run, instead of combinations of values set in advance.

Another way to fine-tune your system is to try to combine the models that perform best. If you aggregate the predictions of a group of predictors (such as different regression models), you will often get better predictions than with the best individual predictor. The *Ensemble Methods* allows us to combine several predictors from different trained models to reach an even better predictor.

3. Performance with Hyperparameter Tuning

The code below performs a grid search cross-validation exercise for our Elastic Net Regression to predict stock returns. The k -fold algorithm randomly splits the training set into 10 different subsamples or "folds", the parameter "n_splits" below. Then, each fold is used as a validation set where the model is trained using the other folds. Then, we have k trained models and we can compute the performance of each fold estimation across them in the test set.

Because the k folds are split randomly, a small value of k can lead to noisy results. This means that each time the procedure is run, a different split of the dataset into k -folds can be implemented, and in turn, the distribution of performance scores can be different, resulting in a different mean estimate of model performance. To solve this, we can repeat the k -fold cross-validation process multiple times, the parameter "n_repeat" below, and report the mean performance across all folds and all repeats. This approach is generally referred to as repeated k -fold cross-validation.

Note: The parameter "random_state" makes sure that the algorithm yields the same reproducible output across multiple function calls.

Once we set up the cross validation technique, we evaluate the model for a pre-determined grid of our hyperparameters in the Elastic Net Regression, the overall degree of regularization, α , and the relative importance of Lasso over Ridge regularization, ρ . We rank models across each repeated k -fold validation using their MSE.

```
In [ ]: from sklearn.model_selection import GridSearchCV, RepeatedKFold

model = ElasticNet()
# define model evaluation method
cv = RepeatedKFold(n_splits=10, n_repeats=3, random_state=1)
# define grid
grid = dict()
grid["alpha"] = [1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 0.0, 1.0, 10.0, 100.0]
grid["l1_ratio"] = [0, 0.01, 0.1, 0.2, 0.5, 0.7, 1]
search = GridSearchCV(model, grid, scoring="neg_mean_squared_error", cv=cv, n_jo
```

```
# perform the search
results = search.fit(X_train, y_train)
# summarize
print("MSE: %.3f" % results.best_score_)
print("Config: %s" % results.best_params_)
```

It turns out that the best model from cross-validation is very close to plain linear regression! The model includes a small degree of ridge regularization and no Lasso regularization.

```
In [ ]: # Train the model
e_net = ElasticNet(alpha=0.001, l1_ratio=0) # Using the above hyperparameters
e_net.fit(X_train, y_train)

# calculate the prediction and mean square error
y_pred_elastic = e_net.predict(X_test)
mean_squared_error = np.mean((y_pred_elastic - y_test) ** 2)
print("Mean Squared Error on test set", mean_squared_error)
```

4. Conclusion

In this lesson, we have described the main methods for regularization and fine-tuning techniques to obtain the best combinations of hyperparameters in the linear regression model.

In the next lesson, we will analyze the main features of classification problems in machine learning.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

MACHINE LEARNING IN FINANCE

MODULE 1 | LESSON 4

SUPERVISED MODELS: CLASSIFICATION CASE STUDY

Reading Time	60 minutes
Prior Knowledge	Logistic regression, Confusion matrix
Keywords	Probability of default

In the last lesson of Module 1, we are going to develop an application of a Machine Learning classification model to predict the probability of default of loan applicants.

1. Predicting the Probability of Default

The probability of default indicates the likelihood that a borrower will be unable to meet their debt payment obligations (either interest or principal payments). Default means that the lender has the legal right to attempt the recovery of their debt by seizing the borrower's assets. The higher the borrower's probability of default, the higher the interest rate charged by the lender, or the lower the amount the borrower would be eligible to obtain from the bank.

We are going to exploit real-world information on default events from a bank in Israel. In this lesson, we will develop a credit scoring model to determine the probability that a loan holder will default training a model from part of the data. We will train the model by looking at historical records of loans held and the individual characteristics of the borrowers (e.g., age, educational level, debt-to-income ratio, and other variables).

Let's read the data.

```
In [ ]: import pandas as pd

df = pd.read_csv("bank.csv")
df.head()
```

The dataset includes a sample of several tens of thousands of previously granted loans. It covers 41,188 records and 10 fields. The data includes an individual identifier and shows whether each loan defaulted or not (\$y=0\$ for no default and \$y=1\$ for default), as well as borrowers' features: age, education level (university degree, high school, illiterate,

basic education, and professional coursework), years with current employer, years in the same home, income, debt-to-income ratio, credit card debt, and other debt.

We will exploit the informativeness that several variables may have at predicting the default behavior of individuals using a logistic regression model. The model will be of use to banks or credit issuers to estimate the probability of default of an individual credit applicant with certain characteristics. A good prediction model will allow the bank to provide better and personalized financial services to customers.

The education variable is categorical with several categories. For estimation of the model, we are going to build a separate indicator variable (or "dummy variable") that shows if an individual has each education level (value of 1) or not (value of zero).

```
In [ ]: cat_vars = ["education"]
for var in cat_vars:
    cat_list = "var" + "_" + var
    cat_list = pd.get_dummies(df[var], prefix=var)
    data1 = df.join(cat_list)
    df = data1
cat_vars = ["education"]
data_vars = df.columns.values.tolist()
to_keep = [i for i in data_vars if i not in cat_vars]
```

```
In [ ]: df_final = df[to_keep]
df_final.drop(["loan_applicant_id"], axis=1, inplace=True)
df_final.head()
```

As usual, we divide the sample into a training set and a test set. The dataset includes more than 41,000 observations that we split in half to determine both samples. Default takes place in roughly 11% of the observations in the data.

```
In [ ]: X, y = (
    df_final.loc[:, df_final.columns != "y"],
    df_final.loc[:, df_final.columns == "y"],
)
print(X.shape, y.shape)
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=int(len(y) * 0.5), shuffle=False
)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
In [ ]: print(df_final["y"].value_counts())
print(
    "Percentage of default: ",
    100
    * df_final["y"].value_counts()[1]
    / (df_final["y"].value_counts()[0] + df_final["y"].value_counts()[1]),
)
```

2. Data Preprocessing

Before training ML algorithms, we should be sure that we are proficient in the information we are working with. Obtaining some prior knowledge about how the data behaves can provide useful guidance to improve the performance of the algorithms.

2.1 Summary Statistics

Computing summary statistics (also called descriptive statistics) is a useful first task that can provide us a first glimpse at the behavior of our data. Averages and medians inform us about the representative value of each variable in our instances. Ranges and variances (or standard deviations) are informative about the dispersion of the variables and may be useful to identify outliers (more on outliers below).

Obtaining the main descriptive statistics of all the variables is very easy with pandas, as shown below. If we take a look at the means or medians of the data as measures of representativeness for each variable, we can see that the individuals in our sample are middle-aged, with long tenures in their jobs, have spent a relatively long period in the same home, and are relatively high-income individuals by Israeli standards, while they are lowly indebted (16% of income). Lastly, the sample is evenly distributed in terms of education across the five levels.

If we take a look at other features of the distribution of input features, such as dispersion, skewness, and kurtosis, we observe that the variables with the most extreme variation are those associated with the amount of debt, both credit and other types of debt. That is, there are several individuals that have extreme levels of debt in absolute terms, relative to most of the individuals.

```
In [ ]: X.describe()
```

```
In [ ]: print(X.skew())
```

```
In [ ]: print(X.kurt())
```

Let's visualize the different aspects of the distributions of each input feature to have a better understanding of the data and explore which variables are more related to default. We are going to plot the distributions of each variable on the same scale. Thus, for better visualization, we are going to standardize the data, i.e., for each input feature, we subtract its mean and divide by its standard deviation. Remember that this is one of the rescaling options we mentioned in Lesson 3.

On top of the standardization, we plot the distribution of each variable separately for those instances where we observe a default and where we do not observe it. Notice in the figure below that four variables display the most significant differences in their distributions between non-defaulting and defaulting individuals: household income, debt-to-income ratio, credit card debt, and other debt. As suggested by the descriptive statistics shown above, the level of credit card debt and other types of debt include individuals with very extreme values.

Notice also that the education attained is does not seem relevant as a default predictor since all the distributions are more or less similar across all education levels. Given this observation we may opt to exclude these variables from our model since they seem to provide little predictive ability compared to the other features. However, we should keep in mind also that there may be relevant non-linear interactions among variables that may help to predict the outcome of interest, despite seeming individually irrelevant at first. Although our Logistic Regression model will ignore the interactions between variables, we should take into account this possibility in more advanced ML models.

```
In [ ]: import matplotlib.pyplot as plt
import seaborn as sns

X_mean = X.mean()
X_std = X.std()
X_std = (X - X_mean) / X_std
X_std["y"] = y
X_std = X_std.melt(id_vars=["y"], var_name="Column", value_name="Normalized")
plt.figure(figsize=(18, 6))
ax = sns.violinplot(data=X_std, x="Column", y="Normalized", hue="y", split=True)
_ = ax.set_xticklabels(X_train.keys(), rotation=90)
```

To further explore the relationship between default and each input feature, we can perform logistic regressions where we only include one feature, on top of the constant bias term. For better visualization and comparability, now we rescale the variables using a min-max method that constrains the value of each variable into the unit interval. Notice again that four variables generate most of the action.

The analysis so far may lead us to think that we could just ignore most of the features. However, notice that the regressions above are very close to linear regressions, which neglect the presence of non-linearities in the data.

```
In [ ]: X_minmax = (X - X.min()) / (X.max() - X.min())
X_minmax["y"] = y
plt.figure(figsize=(18, 6))
g = 1
for colname in X:
    plt.subplot(3, 4, g)
    sns.regplot(
        x=colname,
        y="y",
        data=X_minmax,
        marker="",
        logistic=True,
        fit_reg=True,
        ci=None,
        label=colname,
    )
    plt.title(colname, fontsize="large")
    plt.xlabel("")
    g += 1
    # sns.lmplot(x=colname, y='y', data=X_minmax, markers = "", Logistic = True,
fig = plt.gcf()
fig.tight_layout(pad=1.0)
plt.show()
```

Another important step in data preprocessing is the analysis of correlations between features. Our prediction models may well fail if the ML algorithm is fed with many features that are strongly correlated: The optimization algorithms will face difficulties at disentangling which variable is really relevant for prediction, slowing down the training process.

Below, we display the correlation matrix of the input features, which tell us the degree of covariation between each pair of features. The variables debt-to-income, credit-card debt, and other debt have correlation coefficients above 0.5 among them. Although these correlations are not a big deal, we may have to verify the performance of ML models that exclude at least one of these variables.

```
In [ ]: X.corr()
```

In practice, some applications may involve a number of predictors that make it unfeasible to look at these simple metrics for all the variables. However, minimal verification is recommended. To further ease the analysis:

- Focus on a subset of predictors that a priori should be more relevant when predicting the outcome that we want to predict, e.g., income and debt levels seem the most reasonable in our application here.
- Track outliers in the summary statistics (more on this below).

2.2 Missing Data and Outlier Detection

There are mainly two ways to deal with missing data: removal and imputation. Removal is agnostic but costly if one whole instance is eliminated due to the presence of a single missing feature value. Imputation overcomes the loss of information but requires making assumptions, which are often erroneous.

When facing missing data, we can opt to replace it with the medians or means computed over the cross-section of assets. This implies that the missing feature value will be located within the bulk of observed values. If many values are missing, we would substantially alter the original distribution of the feature and we should assess if it makes sense to include it in our data.

In time series contexts, with views towards backtesting and prediction, the most simple imputation comes from replacing the missing information with the previous value of the time series. If we want to predict returns, however, imputation from past values should be avoided. By default, a superior choice is to set missing return indicators to zero, which is often close to the average or median. Still if a feature is highly autocorrelated, i.e., the correlation between future and past values of the variable are close to one, then imputation from the past can make sense. If not, then it should be avoided.

In time series problems, such as the prediction of future returns, we should also avoid interpolation as a way to fill in missing data. Interpolation over time periods means that we are predicting a future outcome using information that is not available at the time of the prediction. For instance, if earnings figures are disclosed in April and July,

interpolating May and June requires the knowledge of July's earnings. In May, we will not have information about July's earnings, so we cannot make predictions. In this setup, resorting to past values is a better way to go.

Sometimes, we may also detect values of input features that are extremely different from most of the data. We can opt for several heuristic methods that deal with such situations if we are concerned about the effect on the performance of our ML algorithms. These methods involve setting thresholds that determine if a value is considered an outlier:

- Any point outside the interval $[\mu - m\sigma, \mu + m\sigma]$ can be deemed an outlier. μ is the mean of the sample and σ the standard deviation. How stringent we are at labeling outliers is modulated by the multiple value m , which usually belongs to the set $\{3, 5, 10\}$.
- If the largest value is above m times the second-to-largest, then it can also be classified as an outlier (the same reasoning applied for the left side of the distribution).
- For a given small threshold q , any value outside the $[q, 1-q]$ quantile range can be considered an outlier. The range for q is usually (0.5%, 5%) with 1% and 2% being the most often used.

Once we have labeled the outliers, we must determine whether to include them or how to include them in our data. If we decide to include them, the most common practice is to replace the value of the outlier with the corresponding upper or lower threshold that we have obtained following either of the methods above.

2.3 Feature Selection and Scaling

If we have at our disposal a large set of predictors, it is reasonable to filter out redundant or unwanted input features. Simple methods include:

- Computing the correlation matrix of all input features and filter variables so that no (absolute) value is above a threshold (say, 0.7) so that redundant variables do not pollute the learning engine.
- Training linear regressions and removing the non-significant variables.
- Using unsupervised methods (clustering or principal components analysis) to retain a reduced number of features to use as inputs in the algorithm.

The methods above may overlook nonlinear relationships. Another approach would be to fit a decision tree (or a random forest) and retain only the features that have a high variable importance or exploit an autoencoder architecture. You will learn these methods in other modules and courses.

Lastly, as we stressed in the previous lesson, rescaling is needed in many ML applications. Optimization algorithms will better learn about the relevance of each input feature if they share a common scale among them. In the data analysis we performed above, we already executed to versions of feature re-scaling: normalization and min-max rescaling in the unit interval. Using `scikitlearn`, below, we again use min-max rescaling, but in the $[-1, 1]$ interval, to train our model.

```
In [ ]: from sklearn.preprocessing import MinMaxScaler

scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train)
X_train = scaler_input.transform(X_train)
X_test = scaler_input.transform(X_test)
```

3. Logistic Model

Now turn to train the prediction model. We do not specify any particular parameters when we define the class `logisticRegr`, meaning that we use the default optimization options. This implies that the model is regularized using ridge regularization, where the parameter α is set to 1. Please review the documentation to understand the different options at hand to refine the training of the model.

Below, we display the trained parameters of the model, which inform us about the effect of each variable on the probability of default. For instance, households with a higher debt-to-income ratio have a greater probability of default, which is intuitive from an economic standpoint. However, other parameters may make less sense at first glance. For instance, older individuals, longer tenures, or university education seem positively associated with default. Notice that this dataset provides the default behavior of individuals *conditional* on that individual having obtained a loan. For instance, older individuals usually have lower levels of debt than younger individuals. Thus, old individuals that have loans outstanding are likely to be in a worse financial condition than relatively similar young individuals. We can make similar interpretations for the remaining trained parameters.

In any case, our core task here is to develop a model that predicts well out of sample, rather than making sense of the relationship between each particular input and the probability of default. Thus, the trained parameters are of lesser importance if we can obtain a good predictive model.

```
In [ ]: from sklearn.linear_model import LogisticRegression

# parameters not specified, all are set to their defaults
logisticRegr = LogisticRegression()
logisticRegr.fit(X_train, y_train)
```

```
In [ ]: print(logisticRegr.intercept_)
for cc in range(len(X.keys())):
    print(X.keys()[cc], logisticRegr.coef_[0, cc])
```

Below, we display the accuracy of the trained model, which is roughly 92.5%. The average probability of default in the sample is 11.3%, so the model seems relatively able to discriminate some of the default cases in the test sample.

```
In [ ]: predictions = logisticRegr.predict(X_test)
```

```
In [ ]: # Use score method to get accuracy of model
score = logisticRegr.score(X_test, y_test)
print(score)
```

Because analyzing the accuracy of the model may be misleading, we are going to obtain additional performance measures that we described in the previous lesson.

First, we obtain and depict the confusion matrix. Notice now that the model fails quite terribly at predicting many of the default events. That is, the model is quite conservative and predicts in most cases that individuals will not default. This yields a low number of "false positives" but a large number of "false negatives."

```
In [ ]: from sklearn import metrics

cm = metrics.confusion_matrix(y_test, predictions)
plt.figure(figsize=(9, 9))
sns.heatmap(cm, annot=True, fmt=".3f", linewidths=0.5, square=True, cmap="Blues_")
plt.ylabel("Actual label")
plt.xlabel("Predicted label")
all_sample_title = "Accuracy Score: {0}".format(score)
plt.title(all_sample_title, size=15);
```

The precision of class 1 in the test set tells us out of all the default predictions made by the model, 98% were actually "bad" loan applicants. The recall of class 1 in the test set indicates the proportion of the defaulting loan applicants that our model has managed to identify as a "bad" loan applicants. So, the model only managed to identify 34% of "bad" loan applicants.

```
In [ ]: from sklearn.metrics import classification_report

print(classification_report(y_test, predictions))
```

Below, we depict the ROC curve, which tells us how the model's classification performs when we wish to increase recall, at the expense of increasing the proportion of true positives. The Area Under the Curve (AUC) is well below 1. Notice that a 0% of false positives can be obtained by only accepting around a 40% of true positives, which is close to what the trained model yields with threshold probabilities of 50%. Achieving a higher share of true positives can be obtained only at the cost of predicting more "good" individuals as bad. Thus, in an imperfect model like this, we can use our own discretion to choose if we prefer a model that over- or under-estimates the probability of default.

```
In [ ]: from sklearn.metrics import roc_auc_score, roc_curve

sns.set(style="whitegrid", color_codes=True)
logit_roc_auc = roc_auc_score(y_test, logisticRegr.predict(X_test))
fpr, tpr, thresholds = roc_curve(y_test, logisticRegr.predict_proba(X_test)[:, 1])
plt.figure()
plt.plot(fpr, tpr, label="Logistic Regression (area = %0.2f)" % logit_roc_auc)
plt.plot([0, 1], [0, 1], "r--")
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
```

```
plt.title("Receiver operating characteristic")
plt.legend(loc="lower right")
plt.show()
```

A simple alternative to improve the prediction of default by the model is to increase its complexity using powers of the input features, as we did in Lesson 1. Adding this, we can improve the fit of the model by taking into account non-linearities in the relationship between default and the input features. For instance, the probability of default may not increase or decrease monotonically with age; it may well be lower for middle-aged individuals relative to younger or older ones.

In the block of code below, we train a logistic regression model where the input features include up to the 6th power of each input feature. In the optimization, we increase the maximum number of iterations relative to the default value of 100 due to the increased complexity of the model. Moreover, we also adjust the regularization parameter, setting "C" to 2.5. This parameter is an inverse indicator of the strength of regularization, the parameter α in Lesson 2, meaning that we are regularizing less than in the default case where it takes the value of 1. Feel free to check the documentation of the function and play around with the parameters of the model to see its performance under different settings.

The results we show below suggest a considerable improvement from adding powers of the input features. The recall increases up to 54%, which is still quite bad, but much better than the previous model. This suggests that non-linearities are relevant to improve the predictions of the model.

The relevance of these non-linearities is difficult to gauge from direct observation of the data. Other tools in machine learning, such as neural networks, are able to "identify" them in a flexible manner without the need to strictly impose functional forms and improve upon the learning ability of the models. You will study these topics in later modules.

```
In [ ]: import numpy as np

# Redefine the input feature matrix to include powers of each feature
Xpoly = X
for pp in range(2, 6):
    Xpoly = np.concatenate((Xpoly, np.power(X, pp)), axis=1)
X_train, X_test, y_train, y_test = train_test_split(
    Xpoly, y, test_size=int(len(y) * 0.5), shuffle=False
)
# Scale the features
scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train)
X_train = scaler_input.transform(X_train)
X_test = scaler_input.transform(X_test)
# Set up Logistic Regression
logisticRegr = LogisticRegression(C=2.5, max_iter=500)
logisticRegr.fit(X_train, y_train)
# Display coefficients
print(logisticRegr.intercept_)
print(logisticRegr.coef_)
# Compute accuracy
```

```

predictions = logisticRegr.predict(X_test)
score = logisticRegr.score(X_test, y_test)
print(score)
# Display confusion matrix and other indicators
cm = metrics.confusion_matrix(y_test, predictions)
plt.figure(figsize=(9, 9))
sns.heatmap(cm, annot=True, fmt=".3f", linewidths=0.5, square=True, cmap="Blues_")
plt.ylabel("Actual label")
plt.xlabel("Predicted label")
all_sample_title = "Accuracy Score: {0}".format(score)
plt.title(all_sample_title, size=15)
print(classification_report(y_test, predictions))

```

```

In [ ]: logit_roc_auc = roc_auc_score(y_test, logisticRegr.predict(X_test))
fpr, tpr, thresholds = roc_curve(y_test, logisticRegr.predict_proba(X_test)[:, 1])
plt.figure()
plt.plot(fpr, tpr, label="Logistic Regression (area = %0.2f)" % logit_roc_auc)
plt.plot([0, 1], [0, 1], "r--")
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Receiver operating characteristic")
plt.legend(loc="lower right")
plt.show()

```

4. Conclusion

We have covered a complete case study to predict the probability of default using the logistic model. In the next modules, you will learn further useful machine learning techniques.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

MACHINE LEARNING IN FINANCE

MODULE 1 | LESSON 3

SUPERVISED MODELS: CLASSIFICATION

Reading Time	60 minutes
Prior Knowledge	Linear regression, Hyperparameters
Keywords	logistic regression, confusion matrix

In the last two lessons, we explored the supervised linear regression model. In this lesson, we will turn our attention to classification methods.

1. Introduction to Classification Models

In Lesson 1, we mentioned that the most common supervised learning tasks are regression (predicting values) and classification (predicting classes). In the first two lessons, we explored a regression task, predicting stock returns, using linear regression, and using regularization methods. Now, we will turn our attention to classification methods.

1.1 Time Series Momentum: Thresholds

Let's consider the return prediction problem we studied in Lesson 1. Instead of trying to predict the realization of returns, we are now only interested in predicting whether the market is going up or down. Thus, we are going to classify the market into categories of positive return and negative return.

Next, as we did in previous lessons, we upload the necessary information to train and evaluate the prediction model.

```
In [ ]: import numpy as np
import yfinance as yf

# Getting historical market data from SPY (ETF) (SPY)
df = yf.download("SPY", start="2000-01-01", end="2022-01-01")

df["Ret"] = df["Adj Close"].pct_change()

name = "Ret"
df["Ret10_i"] = (
    df[name].rolling(10).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 10) - 1
)
```

```

df["Ret25_i"] = (
    df[name].rolling(25).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 25) - 1)
)
df["Ret60_i"] = (
    df[name].rolling(60).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 60) - 1)
)
df["Ret120_i"] = (
    df[name].rolling(120).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 120) - 1)
)
df["Ret240_i"] = (
    df[name].rolling(240).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 240) - 1)
)

del df["Open"]
del df["Close"]
del df["High"]
del df["Low"]
del df["Volume"]
del df["Adj Close"]

df = df.dropna()
df.tail(10)

```

```

In [ ]: df["Ret25"] = df["Ret25_i"].shift(-25)
df = df.dropna()
df.tail(10)

```

We now transform our previous continuous label y_t into a $\{-1, 1\}$ variable that indicates whether the market has gone, respectively, down or up over a 25-day interval.

```

In [ ]: df["Output"] = df["Ret25"].apply(np.sign)
del df["Ret25"]
df.tail(10)

```

```

In [ ]: df.describe()

```

Split the information into a training set and test set. Below, we display the times series of the label we want to predict.

```

In [ ]: X, y = df.iloc[:, 0:-1], df.iloc[:, -1]
print(X.shape, y.shape)
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=int(len(y) * 0.5), shuffle=False
)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)

```

```

In [ ]: import matplotlib.pyplot as plt

plt.figure(figsize=(15, 7))
plt.plot(
    df.index,
    np.cumsum(y),
    label="Cumulative ups (+1) and downs (-1) of SPY over 25 days",
)
legend = plt.legend(loc="upper left")

```

```
# plt.ylim([-1.25,1.25])
plt.show()
```

1.2 Scaling

One important step before training an ML model involves data pre-processing. We will cover several techniques in the last module, but here, we are going to introduce one of the most important steps: *feature scaling*. Machine Learning algorithms perform badly when the input features have very different scales, which tends to slow down and worsen the performance of the optimization algorithms. There are two common ways to get all attributes to have the same scale: min-max scaling and standardization.

Min-max scaling (many people call this normalization) is quite simple: Values are shifted and rescaled so that they end up ranging from 0 to 1. We do this by subtracting the min value and dividing by the max minus the min. Scikit-Learn provides a transformer called `MinMaxScaler` for this. Formally, each feature x_j if instance i , $x_j^{(i)}$, is transformed as:
$$x_j^{(i)} \mapsto \frac{x_j^{(i)} - \min_j}{\max_j - \min_j}$$
 where $\min_j$ and $\max_j$ denote, respectively, the minimum and maximum values of the j th feature.

Standardization is quite different: First, it subtracts the mean value (so standardized values always have a zero mean), and then it divides by the variance so that the resulting distribution has a unit variance. Unlike min-max scaling, standardization does not bound values to a specific range, which may be a problem for some algorithms. Scikit-Learn provides a transformer called `StandardScaler` for standardization.

Another form of feature pre-processing is referred to as *whitening*, in which the axis-system is rotated to create a new set of de-correlated features, each of which is scaled to unit variance. Typically, Principal Component Analysis (a method of Unsupervised ML) is used to achieve this goal.

It is important to fit the scalers to the training data only, not to the full dataset (including the test set). Only then can you use them to transform the training set and the test set (and new data).

The block of code below scales the observations in our data.

```
In [ ]: from sklearn.preprocessing import MinMaxScaler

scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train)
X_train = scaler_input.transform(X_train)
X_test = scaler_input.transform(X_test)
```

2. Logistic Regression Model

Logistic Regression (also called Logit Regression) is commonly used to estimate the probability that an instance belongs to a particular class (e.g., what is the probability that the market will go up?). If the estimated probability is greater than 50%, then the model

predicts that the instance belongs to that class (called the positive class, labeled "1"), or else it predicts that it does not (i.e., it belongs to the negative class, labeled "0"). This makes it a binary classifier.

Suppose that we have a training sample with instances where the labels y simply denote if the market has gone up $y=1$ or gone down $y=0$. Just like a linear regression model, a logistic regression model computes a weighted sum of the input features (plus a bias term), but instead of outputting the result directly like the Linear Regression model does, it outputs the logistic of this result:
$$\hat{p}^{(i)} = \sigma(x^{(i)\top} \theta)$$
 where $\sigma(\cdot)$ is the sigmoid function defined as:
$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$
 Then, the model's prediction from the Logistic Regression is given by:
$$\hat{y}^{(i)} = \begin{cases} 0 & \text{if } \hat{p}^{(i)} < 0.5 \\ 1 & \text{if } \hat{p}^{(i)} \geq 0.5 \end{cases}$$
 Notice that the Logistic Regression then predicts 1 if $x^{(i)\top} \theta$ is positive and 0 otherwise.

2.1 Training and Cost Function

The objective of training is to set the parameter vector θ so that the model estimates high probabilities for positive instances ($y^{(i)} = 1$) and low probabilities for negative instances ($y^{(i)} = 0$). This idea is captured by the following cost function:
$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$
 This cost function makes sense because $-\log(\hat{p}^{(i)})$ grows very large when $\hat{p}^{(i)}$ approaches 0, so the cost will be large if the model estimates a probability close to 0 for a $y^{(i)} = 1$, and similarly for $y^{(i)} = 0$ if the negative when $\hat{p}^{(i)}$ approaches 1. Indeed, in Econometrics, the previous function is nothing more than the log-likelihood function for a Logit model.

While there is no closed form for the vector θ that maximizes the cost function above, we know that the function is convex and, for instance, Gradient Descent is guaranteed to find the global minimum as long as the learning rate is small enough and we wait for enough time. Thus, if we opt to use the Gradient Descent algorithm (either Batch, Mini-batch, or Stochastic) we would use the following partial derivatives from the cost function:
$$\frac{\partial}{\partial \theta_j} J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} - \sigma(x^{(i)\top} \theta) \right] x_j^{(i)}$$
 Just like in the linear regression, logistic regression can be regularized using the ridge and lasso penalties we described in Lesson 2. Scikit-Learn actually adds an ℓ_2 (Ridge) penalty by default.

Let's train our stock market prediction model using time series information with a Logistic Regression.

Note: If we were interested in playing with the regularization strength, we should include as parameter the input "C" in the class `LogisticRegr` defined below, where "C" is an inverse of the regularization strength.

```
In [ ]: from sklearn.linear_model import LogisticRegression

# all parameters not specified are set to their defaults
logisticRegr = LogisticRegression()
logisticRegr.fit(X_train, y_train)
```

```
In [ ]: predictions = logisticRegr.predict(X_test)
```

```
In [ ]: # Use score method to get accuracy of model
score = logisticRegr.score(X_test, y_test)
print(score)
```

The model is able to predict the actual outcome 72% of the time. It seems a quite considerable figure given the simplicity of the model, right? Below, we show that this figure can be quite misleading.

Before describing in detail the performance measures of classifiers, we first describe a simple generalization of logistic regression to a multiclass setup.

2.2 Softmax Regression

The logistic regression model generalizes to multiple classes, without having to train and combine multiple binary classifiers. This is called *Softmax Regression* or *Multinomial Logit Regression*. For instance, imagine that we want to add a new category to our return prediction model that captures small movements in the returns of the market, say in the $[-.5\%, +.5\%]$ range. Thus, we can train a model that predicts the probability that the market return is "high" ($>.5\%$), low ($<-.5\%$), or "intermediate".

The Softmax Regression model first computes a score for each instance i and category c , $x^{(i)} \theta^{(c)}$, and then estimates the probability of each class by using the softmax function. Notice that each category has a specific vector of parameters, $\theta^{(c)}$, with dimension equal to the number of input features, plus a bias term.

Then, if we have C different categories, the estimated probability of category c for instance i is given the exponential of its score, normalized by the sum of the exponential of the scores across categories.
$$\widehat{p}_{c^{(i)}} = \text{Softmax}_c \left(x^{(i)}, \Theta \right) = \frac{\exp \left(x^{(i)} \theta^{(c)} \right)}{\sum_{j=1}^C \exp \left(x^{(i)} \theta^{(j)} \right)}$$
 where Θ collects all the parameters of the model. The model prediction $\widehat{y}^{(i)}$ will be the category with the highest score $x^{(i)} \theta^{(c)}$.

In analogy with the logistic regression, the cost function to train the parameters of the model can be defined as:
$$J(\Theta) = - \frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C \mathbb{1}_{\{y^{(i)}=c\}} \log \left(\widehat{p}_{c^{(i)}} \right)$$

$$-\frac{1}{m} \sum_{c=1}^C y^{(i)} \log(\widehat{p}_c^{(i)})$$
 which is generally called the *cross-entropy* cost function. The corresponding elements of the gradient vector are:

$$-\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} - \text{Softmax}_c(\mathbf{x}^{(i)}, \Theta) \right] \mathbf{x}_k^{(i)}$$
 From which we can use Gradient Descent optimization.

3. Performance Measures in Classification

Evaluating a classifier is often significantly trickier than evaluating a regression model. One simple measure of performance may simply be the frequency with which the classifier is successful at classifying the labels in the test sample, i.e., its *accuracy*. However, this indicator may be misleading. In our stock market prediction model, we found a 72% success rate in predicting whether the stock market is going up or down. However, such predictive power may arise simply from the model predicting most of the time that the stock market is going up, which happens with roughly 72% probability. This is the actual case, as we are going to show below.

Similarly, our classification model may target the prediction of relatively rare classes or events, say a 5% probability event or class. In those instances, even a terrible classifier may achieve high accuracy. For instance, if we always predict that the event is never happening we are going to reach by default a 95% accuracy. To overcome this and improve the selection of classifier models, there are many performance measures available that we outline below.

3.1 Confusion Matrix

A much better way to evaluate the performance of a classifier is to look at the confusion matrix. The general idea is to count the number of times instances of category A are classified as category B. To compute the confusion matrix, you first need to have a set of predictions so they can be compared to the actual targets. Each row in a confusion matrix represents an actual class, while each column represents a predicted class. A perfect classifier would have only true positives and true negatives, so its confusion matrix would have nonzero values only on its main diagonal (top left to bottom right).

In the case of a two-category classifier where 0s are the negative class and 1s are the positive class, we can represent the confusion matrix as:

$$\begin{matrix} & \text{Actual Negatives} & \text{Actual Positives} \\ \text{Predicted Negatives} & \text{True Negatives} & \text{False Positives} \\ \text{Predicted Positives} & \text{False Negatives} & \text{True Positives} \end{matrix}$$

Let's observe the confusion matrix we obtain from our classifier.

```
In [ ]: # import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import metrics

cm = metrics.confusion_matrix(y_test, predictions)
plt.figure(figsize=(9, 9))
sns.heatmap(cm, annot=True, fmt=".3f", linewidths=0.5, square=True, cmap="Blues_")
plt.ylabel("Actual label")
plt.xlabel("Predicted label")
```

```
all_sample_title = "Accuracy Score: {0}".format(score)
plt.title(all_sample_title, size=15);
```

Indeed, the 72% success rate of the model comes from it predicting that the stock market is going up, which takes place with a probability that is close to that success rate. Thus, we can assess that our model is a poor classifier to predict the returns of the stock market.

3.2 Precision and Recall

The extent of the positive predictions can be obtained through the precision of the classifier:
$$\text{Precision} = \frac{TP}{TP + FP}$$
 where TP is the total number of true positives and FP is the number of false positives. That is, the denominator contains the number of times that the classifier has predicted a 1, so precision captures the proportion of accurate predictions over all the 1s that have been predicted by the model.

Precision is usually combined with another metric, recall:
$$\text{Recall} = \frac{TP}{TP + FN}$$
 Recall captures the proportion of accurate predictions over all the 1s that appear in the sample.

Precision and recall are usually combined in a single metric that is named F_1 score, a harmonic mean of both values that gives much more weight to low values. Thus, the F_1 score will only be high if both precision and recall are high:
$$F_1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F_1 score favors classifiers with similar precision and recall. Sometimes, we do not want that. For instance, if we develop a model to detect the probability of a large drop (crash) in the stock market, in which case the category is 1, we may be willing to sacrifice some accuracy, predict the crash when it does not take place, against some recall, do not predict the crash when the crash actually happens.

Unfortunately, we cannot have classifiers with both high precision and high recall at the same time. To see this, notice that models like logistic regression return probabilities rather than discrete outputs. To determine which class the model is predicting, we set a threshold value on the predicted probabilities to distinguish between a positive and a negative class, such as 0 or 5 in the Logistic Regression above. Depending on the threshold value, the predicted class of some observations may change. If a classifier sets a high threshold to increase precision, it is going to increase the number of positive cases that are going to be predicted as negative, reducing recall. Otherwise, if a classifier sets a high bar to increase recall, it is going to be at the cost of predicting more negative cases as positives, reducing precision.

3.3 ROC and AUC

The Receiver Operating Characteristic (ROC) curve is another common tool used with binary classifiers. The ROC plots the recall of a classifier against the *false positive rate*, the proportion of 0s that are classified as 1s. As in the precision-recall trade-off, when a

classifier increases its recall, then it is going to generate more false positives. One way to compare classifiers is to use the Area Under the Curve (AUC), which gives us the area that lies below the ROC of a classifier. For a perfect classifier, the AUC is one. For a purely random classifier, the AUC is 0.5.

What metric should we use to determine what is a good classifier? We should use the F_1 when we care more about the false positives or when the positive class is rare.

The block of code below plots the ROC curve for our logistic regression model. To obtain this, the `roc_curve()` function computes the recall, or True Positive Rate, and the False Positive Rate in the test sample for various threshold values that determine the prediction of the model. Notice how the predictive ability of the model is *among the poorest of the poor*.

```
In [ ]: from sklearn import metrics

fpr, tpr, thresholds = metrics.roc_curve(y_test, predictions)
roc_auc = metrics.auc(fpr, tpr)
display = metrics.RocCurveDisplay(
    fpr=fpr, tpr=tpr, roc_auc=roc_auc, estimator_name="estimation"
)
display.plot()
plt.show()
```

4. Conclusion

With this, we finish the description of basic classification problems in Machine Learning in the context of logistic regression. In Lesson 4, we will develop an application for credit scoring.

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.